

Modern Model Architectures

Block by Block

Learn the basics. Compute the blocks.
Compare modern model architectures.

Ayoub Kirouane

kirouane-ayoub.github.io

Working manuscript · September 2026

A field guide to the whole repository

From your first vector calculation to modern memory, routing, depth, and optimization.

This working manuscript covers the current nano-models repository: all seven model families, every entry in the architecture guide, the configuration presets, and the supporting training and integration code. It starts with basic Python and arithmetic. Vectors, probability, loss, and a tiny transformer are introduced before the modern architectures.

The goal is to calculate a small example first, explain the problem behind a block, compare alternatives, and inspect a small implementation. Ten standard-library workbook lessons print intermediate values; the PyTorch cache lab checks actual repository tensors. Source code remains a linked companion. A book can teach the code more effectively than it can reproduce every line of it.

HOW TO READ THE EVIDENCE

Mechanism explanations, source-paper claims, and proposed experiments are different kinds of evidence. This edition contains no new quality or throughput benchmarks. Exercises are invitations to measure; the repository roadmap records the remaining validation gaps. A full paper-by-paper quantitative audit remains part of release editing.

The seven visual guides and nineteen component diagrams are original, editable HTML/SVG compositions using a custom latte palette and the checking/rendering tools from [EpicInfographics](#). Their numbers are explicit calculations or labelled illustrations.

Author: Ayoub Kirouane · [Portfolio](#)

Scope: Language Models · September 2026

Companion: [nano-models](#) · Editorial baseline: 0c5d189

Contents

A field guide to the whole repository	2
Remember. Compute. Stabilize.	7
Start here: numbers before names	8
A language model makes a prediction	8
Vectors: keep the coordinates together	8
Matrices: turn one vector into another	9
Exponentials and logarithms: two opposite operations	9
From scores to probabilities	10
Loss: put a number on a wrong prediction	10
Three different kinds of memory	11
Run your first calculation	11
Check your understanding	12
Build one tiny transformer by hand	14
Look up the tokens	14
Give positions a way to differ	14
Attention: score, normalize, combine	14
Hide the answer from the model	15
Many heads, one output shape	15
Add the update back	16
Understand a gate before meeting SwiGLU	16
Normalize and assemble the block	17
Convert the result into a next-token prediction	17
Connect every step: one complete prediction	17
Finish the same calculation with a loss	19
Cache what the next step will reuse	20
What should stay true when you change the architecture?	21
Read the code after doing the arithmetic	21
Check your understanding	21
Attention: your model fits. Its history keeps growing.	23
MHA – Multi-Head Attention	23
GQA – Grouped-Query Attention	24
MLA – Multi-Head Latent Attention	24
K-as-V	25
SWA – Sliding Window Attention	26
DSA – DeepSeek Sparse Attention	27
MoBA – Mixture of Block Attention	28
QSA – Qwen Sparse Attention	29
CSA / HCA – Compressed Attention	29
When the right history still gives the wrong signal	30

Gated Attention	30
Attention sinks	31
Logit softcapping	31
Scalable Softmax	31
Differential Attention	32
Try it: predict the cache, then inspect it	32
Make the design decision	33
Remember with a state instead of a list	35
Gated DeltaNet	35
KDA – Kimi Delta Attention	36
Lightning Attention	37
Mamba-2 and Mamba-3	37
ShortConv	38
Hybrid attention (the 3:1 layout)	39
Compare the memory rules	39
Compute it and change one input	40
Keep numbers well scaled and positions distinguishable	41
RMSNorm	41
Sandwich norm	42
OK-norm	43
RoPE	44
NoPE	44
p-RoPE – partial RoPE	44
Choose the intervention point	45
Compute and compare	46
Choose which feed-forward network does the work	48
MoE – Mixture of Experts	48
Sigmoid routing	49
DeepSeek-V4 routing: hash bootstrap, Sqrt(Softplus), clamped SwiGLU	50
Expert-choice routing	50
Shared expert	50
Aux-loss-free load balancing	51
LatentMoE	51
Compute the route	52
Change the paths: memory, residuals, and repeated depth	54
mHC – Manifold-Constrained Hyper-Connections	54
GR – Gated Residual	54
PLE – Per-Layer Embeddings	55
Engram – conditional memory	56
Mixture-of-Depths	57
KV sharing (cross-layer attention)	58

Looped depth – depth-recurrent transformers	59
Parallel block	60
The Gemma recipe	61
MTP – Multi-Token Prediction	61
Combine structures deliberately	62
Compute the differences	63
Debug with a question you can answer	65
Start with three small experiments	65
MTP shift	65
Sinkhorn axis order	66
Hyper-connection symmetry	66
DSA top-k ties	67
KL with a zero target	67
Aux losses in reported metrics	67
Autocast dtype in MoE routing	67
Compressed-entry causality	67
Accelerator built for its device	68
Looped KV cache	68
SWA cache trimmed before attending	69
Trimmed cache that still holds the prefill	69
Expert choice is non-causal	70
fp32 buffers in a half-precision matmul	70
Stale aux loss on a module that did not run	70
Init RNG shifts	70
How a gradient becomes a parameter update	71
Muon	71
Compute one learning step	73
Put the pieces together and compare the models	75
GPT Nano: the reference decoder	75
Qwen Nano: the modern dense baseline	76
DeepSeek Nano: latent attention and experts	76
Qwen-Next Nano: the composition laboratory	76
Looped Nano: parameters versus effective depth	76
Gemma Nano: a dense system of small choices	77
Mamba Nano: a pure recurrent baseline	77
Read the presets as executable hypotheses	77
From a hand calculation to an experiment	79
The minimal training loop	79
Compare one change fairly	79
Configuration is an experimental record	80
Data loading and target alignment	80

Acceleration and precision	80
The Hugging Face adapter	81
Extending the attention zoo	81
A controlled capstone	81
What remains open	82
Repository atlas and paper trail	83
Source map	83
Preset field values	83
Research reading order	88
Edition provenance and credits	89

Remember. Compute. Stabilize.

Read in order once, then use the contents as a reference.

01 / FOUNDATIONS

Start with vectors, matrices, probabilities, and loss. Compute one attention head and trace a tiny decoder.

02 / MEMORY

Attention variants alter storage and access. Recurrent mixers compress history into evolving state. Hybrids spend on both.

03 / SIGNAL AND CAPACITY

Normalization and position shape the signal. Expert routing allocates parameter capacity. Structural mechanisms change the paths information takes.

04 / ENGINEERING

Inspect failure modes, optimizer routing, whole-model recipes, data and training infrastructure. Finish with a controlled experiment and the repository atlas.

Use the worked calculations, comparison tables, and exercises to turn each concept into a question the implementation can answer. For a first pass, choose one model family and trace it end to end. Return to the other components when you can state the constraint they might solve.

Start here: numbers before names

You do not need to have trained a transformer to read this book. You need basic Python, arithmetic, and a willingness to write down intermediate results. We introduce vectors, matrix multiplication, probabilities, and gradients before using them to explain a decoder. The companion repository uses PyTorch; the first workbook uses only Python's standard library.

Our aim is practical: when you meet a new architecture, you should be able to draw what changes, compute a small example, and explain what the change gives up. You will not need to memorize a catalogue of acronyms first.

A language model makes a prediction

Suppose our entire toy vocabulary is `red`, `blue`, and `stop`. A tokenizer maps these symbols to IDs 0, 1, and 2. Real tokenizers often use pieces of words; an ID is an address, not a numerical measure of a word's meaning. ID 2 is not twice as meaningful as ID 1.

Given a prefix, a language model produces one score for every possible next token. These scores are called **logits**. Softmax turns them into probabilities that sum to one. Training changes the model's stored numbers so that useful next tokens receive higher probability. Generation uses the probabilities to choose another token and extends the prefix.

We will use tiny made-up vectors to expose the arithmetic. They are not representations from a trained model and do not claim that one coordinate means one human concept.

Vectors: keep the coordinates together

A vector is an ordered list of numbers. Let $x = [1, 2]$ and $y = [3, 4]$.

Operation	Calculation	Result
Add vectors	$[1+3, 2+4]$	$[4, 6]$
Multiply by a scalar	$2 * [1, 2]$	$[2, 4]$
Elementwise product	$[1*3, 2*4]$	$[3, 8]$
Dot product	$1*3 + 2*4$	11

The elementwise product returns a vector. The dot product returns one number. Confusing these operations is an easy way to misunderstand attention or a gate.

A **tensor** is an array with named dimensions in our explanation: tokens, heads, channels, and so on. Its **shape** tells you the length of each dimension. A tensor of shape $[2, 3, 4]$ contains 24 numbers. Shape alone does not tell you what those numbers mean.

Matrices: turn one vector into another

A matrix is a table of numbers. We use row vectors in the hand calculations. If x has two coordinates and W has two rows and three columns, $x @ W$ has three coordinates. Each output coordinate is a dot product with one column of W .

```
x = [1, 2]
W = [[1, 0, 2],
     [0, 1, 3]]
x @ W = [1*1 + 2*0, 1*0 + 2*1, 1*2 + 2*3]
       = [1, 2, 8]
```

A linear layer computes $x @ W + b$, where b is a bias vector. W and b are **parameters**: stored numbers adjusted during training. This example has six weights and, with bias enabled, three biases. Applying the same layer to ten tokens uses the same parameters ten times; it does not create ten weight matrices.

PyTorch's `Linear` stores the weight with shape $[\text{output_width}, \text{input_width}]$, so its operation is $x @ \text{weight.T} + \text{bias}$. That storage convention differs from our W notation, not from the calculation.

Exponentials and logarithms: two opposite operations

An exponent describes repeated multiplication: $2**3 = 2*2*2 = 8$. The exponential function $\exp(x)$ uses a particular base called e , about 2.71828. It is defined for fractional and negative x too. You can calculate it with `math.exp(x)` in Python.

x	exp(x), approximately
-1	0.3679
0	1
1	2.7183

Exponentials are positive and grow as their input increases. That makes them useful for turning arbitrary scores into positive weights.

The natural logarithm reverses this operation. $\log(y)$ asks: what exponent of e gives y ? Since $\exp(0.693147\dots) = 2$, $\log(2) = 0.693147\dots$. Consequently $\exp(\log(2)) = 2$. In Python, use `math.log(2)` without rounding the result before exponentiating it. Natural logarithms require a positive input; probabilities between zero and one have negative logarithms.

Now softmax's two operations will have a purpose: exponentials make weights positive, and division by their sum makes them add up to one.

From scores to probabilities

Softmax exponentiates each logit, then divides by the sum of those exponentials. For logits $[0, 0, \log(2)]$, the exponentials are $[1, 1, 2]$. Their sum is four, so the probabilities are $[0.25, 0.25, 0.5]$.

```
probability[i] = exp(logit[i]) / sum(exp(logits))
```

In code, subtract the largest logit before exponentiating. This preserves the probabilities because the common factor cancels, and helps avoid numerical overflow. Softmax changes when relative scores change; adding the same constant to every score leaves it unchanged.

PAUSE & COMPUTE

Try before reading on. For logits $[0, \log(3)]$, what are the probabilities? What if we add 100 to both logits?

The answers are $[0.25, 0.75]$ in both cases. A large raw score is not enough to determine its probability; the other scores matter too.

Loss: put a number on a wrong prediction

If the correct next token receives probability p , its cross-entropy loss is $-\log(p)$, using the natural logarithm. For $p = 0.5$, the loss is about 0.6931. For $p = 0.25$, it is about 1.3863. A confident correct answer has a smaller loss. Across valid target positions, the training objective usually averages these values.

Before using a gradient, estimate one with arithmetic. Let a scalar weight be w and a toy loss be $L(w) = 0.5*(w-1)**2$. At $w = 2$, loss is 0.5. At 1.99 it is 0.49005; at 2.01 it is 0.51005. The slope across that small interval is $(0.51005-0.49005)/(2.01-1.99) = 1$.

A **derivative** is the local slope approached as the interval shrinks. For this quadratic, the derivative is $w-1$, so it equals one at $w = 2$. With many weights, collect their individual local slopes into a **gradient**. Each component asks how loss changes if that parameter changes while the others stay fixed. Our positive slope says moving w slightly downward should reduce loss. It does: $w = 1.9$ has loss 0.405.

A **gradient** tells us how a small parameter change affects loss locally. Gradient descent updates a parameter with $\text{new} = \text{old} - \text{learning_rate} * \text{gradient}$. If a scalar parameter is 2, its gradient is 0.5, and the learning rate is 0.1, the next value is 1.95. Backpropagation computes gradients through the chain of operations; an optimizer chooses how to use them.

For softmax plus cross-entropy, the derivative with respect to each logit is $\text{probability} - \text{target}$, where the target vector is one at the correct token and zero elsewhere. We will compute a complete update in the workbook.

Three different kinds of memory

Parameters persist across examples: embeddings, projection matrices, and normalization gains. **Activations** are intermediate values produced while processing an input; training retains what backward needs. **Generation state** contains reusable history such as cached keys and values or a recurrent state.

These are different budgets. A model with fewer parameters can still use more generation memory. A fixed-size recurrent state does not mean fixed-size training activation memory.

For a plain numerical array, logical payload is $\text{number_of_elements} * \text{bytes_per_element}$. Four float32 numbers use 16 bytes of payload. Python objects, allocator bookkeeping, and tensor views can require or retain additional storage. We will always say which quantity we are counting.

Run your first calculation

From the repository root:

```
python book/labs/beginner_workbook.py --lesson basics
python book/labs/beginner_workbook.py --lesson loss
```

These commands need Python 3.11 or newer and no packages, downloads, or GPU. They print intermediate values and check the arithmetic. Read the function with the same name in the workbook, change its toy input, and predict the output before rerunning it.

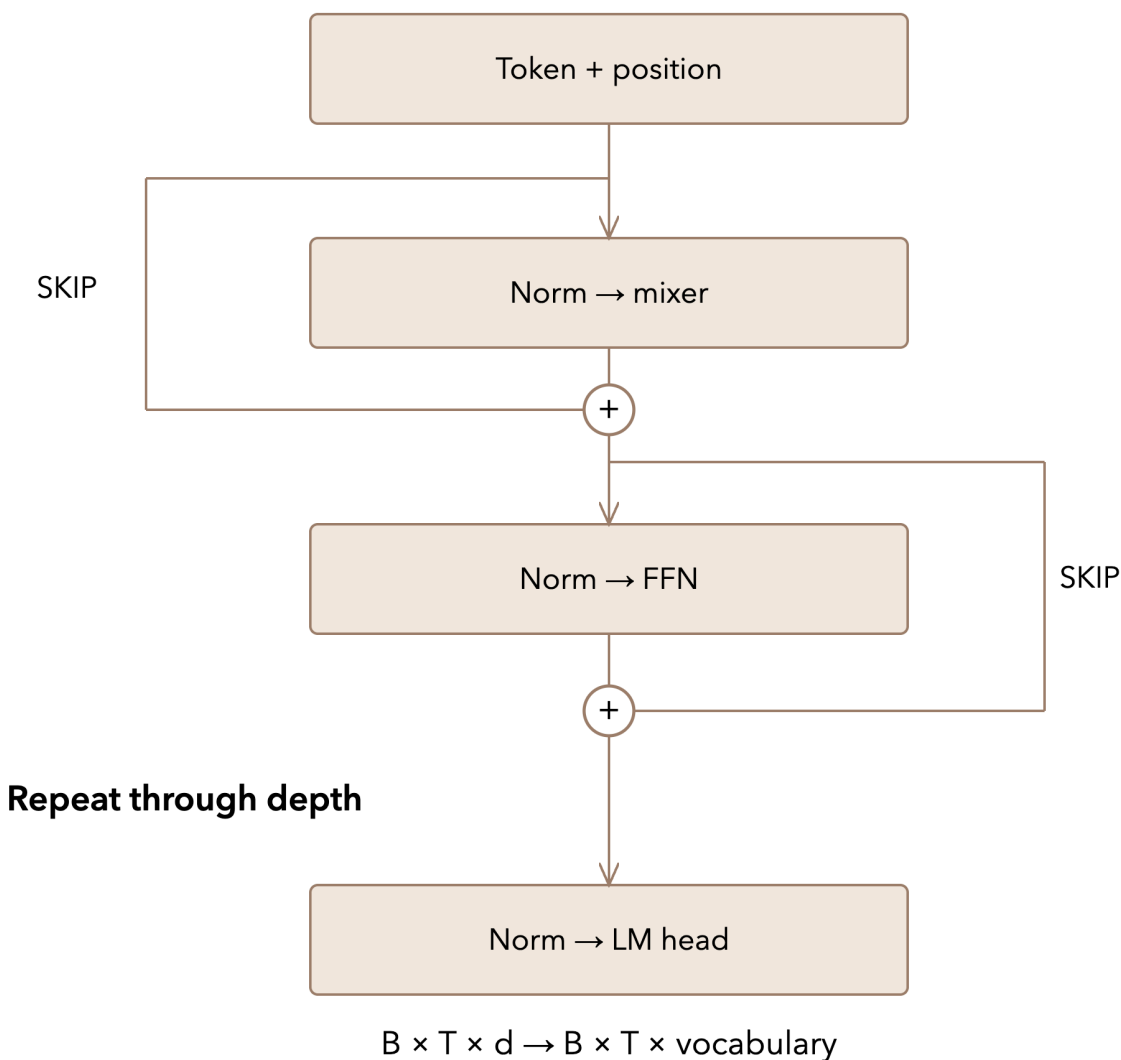
Check your understanding

1. How many scalars are in a [2, 5, 3] tensor?
2. How many weights are in a bias-free layer from width 3 to width 4?
3. If the correct token's probability increases from 0.25 to 0.5, does its loss increase or decrease?
4. Which changes during ordinary generation: the trained weights or the generation state?

Answers: 30; 12; loss decreases from about 1.3863 to 0.6931; generation state changes while the weights stay fixed. Next we will build one decoder pass from these operations.

Follow the Residual stream

A pre-norm decoder is a circuit with two bypasses.



Source: `nano/models/qwen_nano.py`; simplified block

Ayoub Kirouane / nano-models

Build one tiny transformer by hand

A transformer decoder turns token IDs into vectors, lets positions exchange information, transforms each position's features, and predicts the next token. We will follow those operations in that order. Our examples use tiny numbers so you can reproduce every step.

Look up the tokens

An embedding table has one row per token and one column per feature. Give our three-token vocabulary this table:

```
red   -> [1, 0]
blue  -> [0, 1]
stop  -> [1, 1]
```

The input IDs $[0, 1]$ retrieve the rows $[[1, 0], [0, 1]]$. No multiplication by the ID is involved. The table has $\text{vocabulary_size} * \text{width} = 3 * 2 = 6$ learned parameters. These rows are illustrative values, not trained meanings.

A batch is a collection of sequences processed together. With batch B , length T , and width d , IDs have shape $[B, T]$ and embeddings have shape $[B, T, d]$. Padding, if present, needs its own masking and loss rules.

Give positions a way to differ

A sequence has an order. One approach adds a learned position vector to each token vector. Another, RoPE, rotates pairs of coordinates in the queries and keys used by attention. A causal mask also introduces an asymmetry: a position can access earlier tokens but not later ones. The position chapter will show why these mechanisms are not interchangeable.

For the following hand calculation we omit position transformations and pretend the displayed vectors are already the inputs to attention.

Attention: score, normalize, combine

A projection creates three vectors for each token: **query** Q , **key** K , and **value** V . A query scores the available keys using dot products. Softmax converts scores into

weights. The output is a weighted sum of values. Q and K choose how much to read; V supplies what is read.

Consider one head with two coordinates. At the second position, use query $[1, 0]$, keys $[[1, 0], [0, 1]]$, and values $[[2, 0], [0, 4]]$.

```
raw dot products      = [1, 0]
scaled by sqrt(2)    = [0.7071, 0]
softmax weights      = [0.6698, 0.3302]
weighted value sum    = 0.6698*[2, 0] + 0.3302*[0, 4]
                     = [1.3395, 1.3210] approximately
```

The square-root scale uses the head width. It helps control how dot-product magnitudes vary with dimension; it is not a probability normalization. Softmax supplies the sum-to-one property.

Hide the answer from the model

The first position cannot read the second position while learning to predict it. A **causal mask** disallows attention to future keys. We represent a forbidden score with negative infinity before softmax, making its probability zero.

Query position	Key 0	Key 1	Key 2
0	allowed	hidden	hidden
1	allowed	allowed	hidden
2	allowed	allowed	allowed

At position 0 in our example, only value $[2, 0]$ is visible, so its attention weight is one and the output is $[2, 0]$. At position 1, the two-weight calculation above is legal. A row with every key hidden is invalid for ordinary softmax; at least one valid destination must remain.

Run the same calculation:

```
python book/labs/beginner_workbook.py --lesson attention
```

Run again with `--value-scale 2` to change the second value to $[0, 8]$. The second coordinate of the second output doubles, but the weights stay unchanged because Q and K did not change. Changing a key can instead change both weights.

Many heads, one output shape

A head is one set of query/key/value channels. Several heads let the layer form different weighted combinations. With width $d = 8$ and two heads, each head has width 4. Splitting $[B, T, 8]$ into heads yields $[B, 2, T, 4]$ after rearranging dimensions. Combining the heads returns width 8, followed by an output projection.

This does not mean each head has a fixed human-readable role. GQA changes the number of stored key/value heads while keeping the query heads; that distinction will drive the next chapter's cache calculation.

Add the update back

A residual connection adds a sublayer's result to its input. If $x = [1, 2]$ and its attention update is $[0.5, -0.5]$, the residual becomes $[1.5, 1.5]$. The sublayer can contribute a change without replacing the entire representation.

A feed-forward network, or FFN, then transforms each position separately using the same weights at every position. A small plain FFN might compute:

```
x           = [1, -2]
expand(x)   = [1, -2, -1]
ReLU(expand) = [1, 0, 0]
project_down = [1, 0]
```

Here ReLU replaces negative coordinates with zero. Production blocks often use GELU or gated activations. The important distinction is that attention mixes positions, while the FFN mixes features at a position.

For a bias-free plain FFN with width d and hidden width h , the two matrices have $2*d*h$ weights. A gated FFN has two input projections and one output projection, giving $3*d*h$. With $d = 4$ and $h = 8$, those counts are 64 and 96. Equal hidden width is not equal parameter budget.

Understand a gate before meeting SwiGLU

A gate multiplies one signal by another. The sigmoid function $1 / (1 + \exp(-x))$ maps a score to a number between zero and one. At score zero it gives 0.5.

Multiplying a value of 4 by that gate gives 2.

SwiGLU uses the related SiLU function, $\text{silu}(x) = x * \text{sigmoid}(x)$, on one learned projection and multiplies it elementwise with another projection:

```
SwiGLU(x) = down(silu(gate_projection(x)) * up_projection(x))
```

The word gate here does not mean its value stays between zero and one: SiLU can be negative or greater than one. For a scalar gate-projection value of 1 and up-projection value of 2, the product is about $0.7311 * 2 = 1.4621$ before the down projection.

GeGLU uses GELU in the gate branch instead. GELU is a smooth activation commonly written $x * \Phi(x)$, where Φ is the standard normal cumulative probability. For this first pass, remember the structural difference from a plain FFN: two learned input branches, an elementwise product, then one output projection. The choice of activation is a separate decision.

Normalize and assemble the block

Normalization controls a vector's scale before or after another operation. RMSNorm divides by root-mean-square magnitude and applies learned gains; LayerNorm also subtracts the mean. We will calculate them in the normalization chapter.

A pre-norm decoder block is:

```
u = x + attention(norm1(x))
y = u + ffn(norm2(u))
```

Read top to bottom. The FFN sees the representation after the attention update. Both sublayers return the residual width so addition is valid. Stack blocks to let successive layers transform the representation further.

Convert the result into a next-token prediction

A vocabulary head maps the final hidden vector to one logit per token. If it reuses the embedding table, the weights are **tied**. For our table and hidden vector $[1, 2]$, the logits are $[1, 2, 3]$: take the dot product with each table row. Softmax gives approximately $[0.0900, 0.2447, 0.6652]$.

For a token sequence $[\text{red}, \text{blue}, \text{stop}]$, the two valid training pairs within the snippet are $\text{red} \rightarrow \text{blue}$ and $\text{red}, \text{blue} \rightarrow \text{stop}$. Inputs are $[\text{red}, \text{blue}]$; targets are $[\text{blue}, \text{stop}]$. The target at position i is the next token, not the input at i . Predicting an input already supplied to the model can make a broken training setup look successful.

During training, all known positions can be processed together with a causal mask. During generation, choose a next token, append it, and repeat. Greedy choice takes the largest probability; sampling draws from the distribution. We will focus on architecture rather than sampling strategies.

Connect every step: one complete prediction

We have practiced the operations separately. Now keep one set of weights and pass the output of each operation into the next. Use the snippet $[\text{red}, \text{blue}, \text{stop}]$: input IDs are $[0, 1]$, and target IDs are $[1, 2]$. Our two-position decoder has one block, one head of width two, a width-three ReLU FFN, and a tied vocabulary head. All biases are zero. There is no dropout or explicit position encoding; attention is causal.

These are all its weight matrices, using row-vector multiplication:

```

E = [[1, 0],      W_Q = W_K = W_O = [[1, 0],
    [0, 1],      [0, 1]]
    [1, 1]]

W_V = [[2, 0],    W_up = [[1, 0, 1],
    [0, 4]]        [0, 1, -1]]

W_down = [[0.5, 0],
    [0, 0.5],
    [0.25, -0.25]]

```

E is the embedding table, whose rows are ordered red, blue, stop. W_Q, W_K, and W_O are identity matrices: they leave their input unchanged. Each normalization divides a vector by $\sqrt{\text{mean}(x*x) + 1e-8}$ with gains fixed at one. This is RMSNorm. For example, normalizing [1,0] gives approximately [1.414214,0]. We normalize before both sublayers and once before the head.

The full calculation is:

```

x = E[input_ids]
a = rmsnorm(x)
Q, K, V = a @ W_Q, a @ W_K, a @ W_V
scores = Q @ K.T / sqrt(2)
weights = softmax(causal_mask(scores)) # separately for each row
u = x + (weights @ V) @ W_O
b = rmsnorm(u)
f = relu(b @ W_up) @ W_down
y = u + f
z = rmsnorm(y)
logits = z @ E.T
probabilities = softmax(logits) # separately for each row

```

Normalization also acts separately on each row. Keep full precision when computing; the following tables round only the displayed answers. Each column follows one position through the same block.

Stage	Position 0: red	Position 1: blue
Embedding x	[1, 0]	[0, 1]
Q and K	[1.414214, 0]	[0, 1.414214]
V	[2.828427, 0]	[0, 5.656854]
Visible scores	[1.414214]	[0, 1.414214]
Attention weights	[1]	[0.195570, 0.804430]
Attention output	[2.828427, 0]	[0.553156, 4.550541]
Residual u	[3.828427, 0]	[0.553156, 5.550541]

Position 0 has only one visible key. Position 1 mixes both values. For example, its first attention-output coordinate is $0.195570... * 2.828427... + 0.804430... * 0 = 0.553156...$. The residual adds its original embedding [0,1], giving u's second coordinate $4.550541... + 1$.

Continue from u	Position 0	Position 1
Normalize u	[1.414214, 0]	[0.140243, 1.407243]
FFN expansion	[1.414214, 0, 1.414214]	[0.140243, 1.407243, -1.267000]
After ReLU	[1.414214, 0, 1.414214]	[0.140243, 1.407243, 0]
FFN output f	[1.060660, -0.353553]	[0.070122, 0.703621]
Residual y = u + f	[4.889087, -0.353553]	[0.623278, 6.254163]
Final norm z	[1.410530, -0.102002]	[0.140243, 1.407243]

At position 0, the FFN's first output coordinate is $1.414214... * 0.5 + 0 * 0 + 1.414214... * 0.25 = 1.060660...$. At position 1, ReLU removes the negative third feature before projection. No hidden vector is replaced with an unrelated example.

Finish the same calculation with a loss

The tied head takes a dot product with each row of E. Given $z = [a, b]$, the three logits are therefore $[a, b, a+b]$ in red/blue/stop order.

Output	Position 0	Position 1
Logits	[1.410530, -0.102002, 1.308528]	[0.140243, 1.407243, 1.547486]
Probabilities	[0.470948, 0.103774, 0.425279]	[0.115810, 0.411145, 0.473045]
Correct target	blue, ID 1	stop, ID 2
Target probability	0.103774	0.473045
Negative log probability	2.265541	0.748565

Average the two losses: $(2.265541... + 0.748565...)/2 = 1.507053...$. These arbitrary weights are not trained, so a poor prediction is expected. The point is that every probability now comes from the input IDs through a fully specified computation.

Run `python book/labs/beginner_workbook.py --lesson decoder`. After the warm-up examples, its **CONNECTED DECODER** section prints every stage above. It also changes the second input and checks that the first position's logits stay unchanged. Finally, the lesson perturbs one actual weight, `w_down[0][0]`, and reruns the entire calculation. A central finite difference estimates its loss gradient as approximately -0.0122033. With learning rate 0.1, that weight moves from 0.5 to 0.5012203 and mean loss falls from 1.5070530 to 1.5070381. Other weights stay fixed. This is one numerical-gradient step in the toy decoder; automatic backpropagation would calculate derivatives efficiently for all trainable parameters. It is not a language-model training benchmark.

Cache what the next step will reuse

Prefill processes the prompt. **Decode** adds new tokens. An attention layer can cache old keys and values because causal earlier representations do not change as future tokens arrive. New queries read those entries without recomputing the earlier projections.

With batch B , layers L , cached tokens T , KV heads H , head width d_h , and s bytes per element:

$$M_{KV} = 2BLTHd_h s$$

For $B = 1$, $L = 2$, $T = 3$, $H = 1$, $d_h = 2$, and float32 $s = 4$, the payload is 96 bytes. Double T and it doubles. This counts conventional retained K/V only, not weights, training activations, or temporary tensors.

Always reset generation state before a new independent sequence. Otherwise the next example can read the previous example's history.

What should stay true when you change the architecture?

Check	What to do	What a failure suggests
Shape	Track every dimension	A wrong reshape or projection
Causality	Change a future token	Earlier outputs must not change
Cached equivalence	Compare full and incremental evaluation	Missing state or wrong position offsets
Learning	Inspect gradients and an actual update	A disconnected path or missing loss

Use numerical tolerances for floating-point comparisons. Disable stochastic behavior such as dropout when checking cached equivalence. The companion model checks are more detailed than the hand examples.

Read the code after doing the arithmetic

Run the standard-library lesson for embeddings, the FFN, and the output head:

```
python book/labs/beginner_workbook.py --lesson decoder
```

Then, in a configured PyTorch environment, inspect `GPTNano` in `gpt_nano.py` and run its existing check:

```
python -m nano.models.gpt_nano --self-check
```

That repository command is a follow-up exercise, not a claim that a full model-training experiment was run for this edition. Compare with `qwen_nano.py` after the architecture chapters.

For the gated FFN's research background, see [GLU Variants Improve Transformer](#). For tied embeddings, see [Using the Output Embedding to Improve Language Models](#). First trace what each operation changes; then read the papers' experiments.

Check your understanding

1. For $B = 2$, $T = 3$, and $d = 4$, what is the embedding output shape?
2. At query position 2, how many keys can full causal attention read?
3. Does tying a vocabulary head remove its matrix multiplication?
4. Why is a future-token perturbation a stronger causality check than merely inspecting the final attention mask?

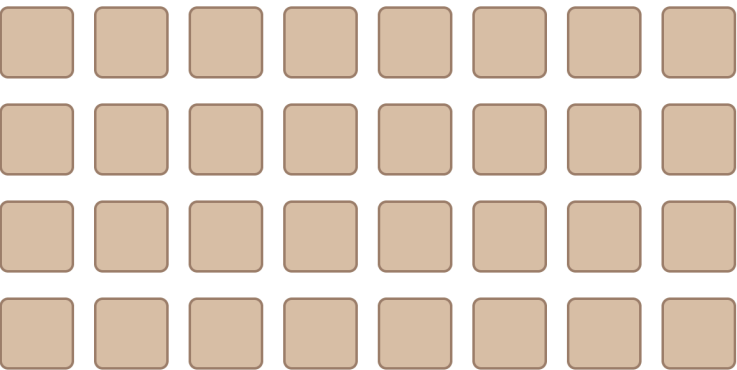
Answers: [2, 3, 4]; three including the current key; no, it shares parameters; other operations such as sequence-wide routing can leak future information before the mask. You now have the pieces needed to compare attention designs.

Share the heads. Shrink the cache.

Calculated storage, with the same context and precision.

Multi-head attention

32 query heads · 32 KV heads

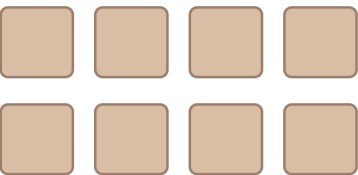


4x
less KV storage

$32 \times 128 \text{ MiB} = 4 \text{ GiB}$

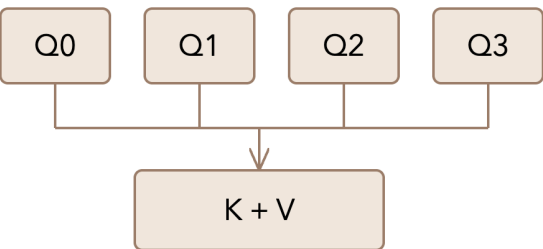
Grouped-query attention

32 query heads · 8 KV heads



$8 \times 128 \text{ MiB} = 1 \text{ GiB}$

One sharing group



Four queries share one KV head.

Attention: your model fits. Its history keeps growing.

We now use the decoder from the previous chapter to compare ways of storing and reading history. Start with the byte counts; then inspect which information each change preserves.

You have a small decoder that can finish a sentence. Now ask it to read a longer document. The weights have not changed, yet generation needs more memory. Every new token leaves keys and values behind for later queries.

Our running model has four layers, four query heads, head width 16, and batch size one. We will count a hypothetical two-byte cache first, then inspect real float32 tensors in the lab. These are teaching dimensions, not a production recipe.

By the end, you should be able to choose between three interventions: **store less per token, read fewer tokens, or store fewer entries**. They solve different problems. A smaller cache is not automatically faster attention, and faster attention is not automatically better retrieval.

PAUSE & PREDICT

Predict first. Double the context from 128 to 256 tokens without changing the model. What happens to the weights, retained KV storage, and the number of query-key pairs in a full prompt pass?

The weights stay fixed. A conventional KV cache doubles. The number of causal query-key pairs grows from 8,256 to 32,896, almost fourfold. We will keep these three costs separate throughout the chapter.

MHA – Multi-Head Attention

Start with the decoder you know. Each query head has its own key and value projections. A query reads the current token and all earlier tokens, so an old fact remains directly addressable.

For our four-layer model, the retained KV payload is:

```

layers, tokens, kv_heads, head_width = 4, 128, 4, 16
bytes_per_scalar = 2
scalars = 2 * layers * tokens * kv_heads * head_width
cache_bytes = scalars * bytes_per_scalar
print(cache_bytes // 1024) # 128 KiB; batch size one

```

The leading two counts K and V. This excludes weights, activations, allocator overhead, and temporary attention tensors. At 256 tokens the same calculation gives 256 KiB. Nothing is broken: full-history attention is doing exactly what we asked.

The first design question is whether every query head needs a separate stored history.

Read the baseline: [MultiHeadAttention](#), introduced in *Attention Is All You Need* (2017).

GQA – Grouped-Query Attention

Keep all four query heads, but let pairs of them share K and V. We now store two KV heads per layer. Our 128-token cache falls from 128 KiB to **64 KiB** while the number of query heads stays four.

This is GQA's compromise: queries remain distinct, but their stored key/value representations are shared within a group. One KV head is the more aggressive multi-query endpoint.

Try the next step mentally. If we use one KV head, the payload becomes 32 KiB. That arithmetic says nothing about whether the trained model will retain the same quality. Sharing imposes a representational constraint; it also leaves each query attending across the sequence.

When reading the GQA paper, notice that its recipe includes converting and uptraining existing multi-head checkpoints. Our cache calculation tests neither that conversion recipe nor its quality results.

Code entry point: [nano/attention_zoo.py](#). **Paper:** *GQA: Training Generalized Multi-Query Transformer Models* (Google, 2023), [arXiv:2305.13245](#).

MLA – Multi-Head Latent Attention

GQA shares stored heads. Could we instead store a compact code from which distinct heads are reconstructed?

That is the idea behind the latent cache. In this repository, a projection compresses each token into a shared latent vector. Later projections expand the retained vector into per-head K/V for attention. The heads can differ, but all must be recoverable from the same bottleneck.

For our running dimensions, a toy latent width of 16 gives $4 \times 128 \times 16 \times 2 = 16$ KiB. This counts the latent only. Expanded temporary K/V can still be large.

Production MLA designs also have details, including rotary-key storage, that this toy does not reproduce.

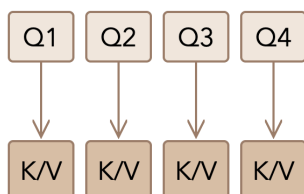
Read the figure as three storage choices. Its smaller head width makes the entries countable by eye. It holds query-head count fixed while changing what survives between decode steps. The tradeoff is sharing versus reconstruction, not a guaranteed ranking of model quality.

Code entry point: [nano/attention_zoo.py](#). **Paper:** [DeepSeek-V2](#) (DeepSeek, 2024).

Different ways to store the same history

Four query heads in every panel; conventional head width = 2.

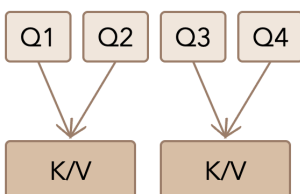
MHA



4 KV heads $\times 2 \times 2$

16 cached scalars / token

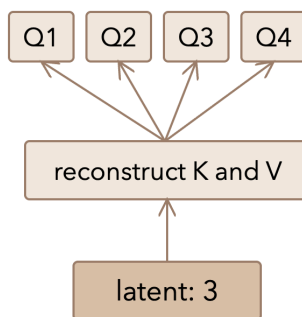
GQA



2 KV heads $\times 2 \times 2$

8 cached scalars / token

MLA



3 cached scalars / token

Illustrative dimensions; latent reconstruction creates temporary tensors.

Trace it. For one token and layer, MHA stores $4 \times 2 \times 2 = 16$ scalars; GQA stores 8. A rank-three shared latent stores 3, before any additional state required by a different MLA design.

Keep in mind. Fewer retained scalars do not imply proportional latency savings. This repo reconstructs K/V and does not implement DeepSeek's decoupled rotary-key cache.

Repository reference: [nano/attention_zoo.py](#)

K-as-V

There is another way to remove stored numbers: use the same representation for matching a query and supplying its value. K-as-V drops the independent value projection.

If our two-head GQA cache stores K and V as one shared tensor, the 64 KiB payload becomes 32 KiB. If an implementation retains two copies of that tensor, the allocation does not halve. This is why inspecting tensor shapes is insufficient.

The constraint is different from GQA's: a head gives up an independently learned value representation. In the repository, kv1 demonstrates this choice; the Gemma model applies it to its global layers. Check actual storage ownership before reporting a memory saving.

Code entry point: kv1 in [nano/attention_zoo.py](#); the global layers of [nano/models/gemma_nano.py](#) cache one tensor. **Paper:** Gemma 4 (Google DeepMind, 2026), [arXiv:2607.02770](#).

SWA – Sliding Window Attention

Sharing heads reduced each token's footprint. History still grows. Suppose our task mostly needs recent context: can we discard older entries entirely?

A sliding window makes visibility depend on distance. With a 32-token window and two KV heads, our running model retains at most **16 KiB** of conventional KV payload once the window fills, provided every layer uses the window and the retained tensors own only that storage.

PAUSE & PREDICT

Predict first. With a three-token window, query 4 can read which positions? Can it directly retrieve a fact stored only at position 0?

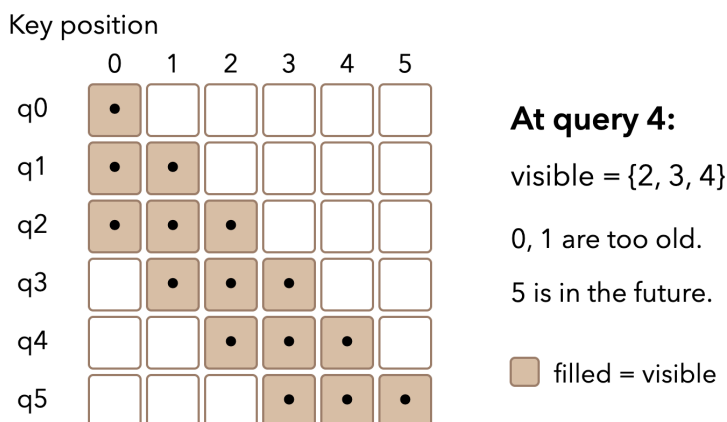
The mask below reveals the answer: positions 2, 3, and 4. Older information may have moved through intermediate representations, but direct access to the old entry is gone. That distinction matters for names, identifiers, and facts mentioned once far back.

A practical hybrid keeps some full-attention layers for long-range access. Its total cache still grows with those global layers. During prefill, do not trim away keys before earlier prompt queries have used them.

Code entry point: [nano/attention_zoo.py](#). **Paper:** Longformer (AI2, 2020); Mistral 7B (Mistral, 2023).

A window is a visibility rule

Toy causal sequence: six positions, window size three.



Rows are queries; columns are keys. Window size is three tokens.

Trace it. At query position 4, the visible keys are 2, 3, and 4. At position 0, only key 0 is available; the window does not require three existing tokens.

Keep in mind. Trimming a long prefill cache before all prompt queries have attended can remove the earlier keys those queries need. Test prompt outputs as well as the next decode step.

Repository reference: [nano/attention_zoo.py](#)

DSA – DeepSeek Sparse Attention

A window fails our next workload: a document names a project near the beginning, then asks for that name near the end. Distance is a poor proxy for relevance.

DSA uses a small learned indexer to score candidate keys, selects top-k, and applies attention to the selected positions. This changes **which entries are read**. It does not, by itself, eliminate the need to retain the history from which future queries will select. Selection also introduces a learning problem. A hard top-k mask does not pass an ordinary gradient through its indices. The repository trains the indexer with an auxiliary objective against a dense attention distribution. Inspect that objective separately: a finite language-model loss does not prove the selector learned anything.

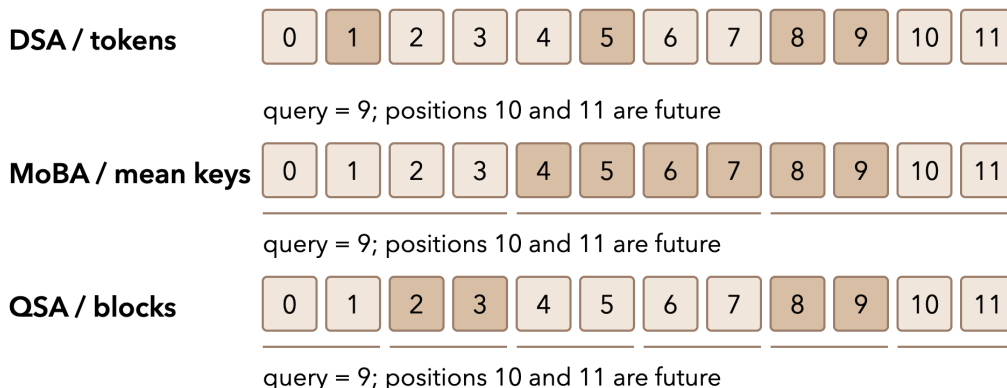
Now predict the failure. What if the relevant token never reaches top-k? The expensive attention stage cannot recover an entry it was never allowed to see. Selection quality is part of model quality.

The comparison figure shows individual-token selection beside two block alternatives. All rows obey causality. This repository uses dense scores followed by masks, so the picture illustrates access rules rather than measured compute savings.

Code entry point: [nano/attention_zoo.py](https://github.com/DeepSeek-AI/DeepSeek-V3/blob/main/nano/attention_zoo.py). **Paper:** DeepSeek-V3.2 (DeepSeek, 2025).

Select tokens, or select groups of tokens

Illustrative choices, not learned scores from a trained model.



Filled cells are selected; underline marks a block. Toy block sizes: 4 and 2.

Trace it. With query 9, the MoBA row selects closed block 4-7 and the visible part 8-9 of its current block. The QSA example uses two-token micro-blocks solely to make the grouping readable.

Keep in mind. Selecting a block never permits its future positions. Dense scoring followed by these masks still performs dense work in the teaching implementation.

Repository reference: [nano/attention_zoo.py](https://github.com/DeepSeek-AI/DeepSeek-V3/blob/main/nano/attention_zoo.py)

MoBA – Mixture of Block Attention

Scoring and selecting individual tokens is not the only option. MoBA divides history into blocks, scores each closed block using its mean key, and reads tokens inside selected blocks. The query's own causal block remains available.

Imagine a four-token block containing one useful identifier and three irrelevant tokens. Selecting that block brings all four along. A larger block can make routing coarser, even when it lowers routing overhead.

The mean-key summary adds no separate learned indexer parameters, although the underlying keys are learned. Test `top-k = 0`: the query must still see its own causal block. This makes the fallback concrete and guards against fully masked rows.

Code entry point: [nano/attention_zoo.py](#). **Paper:** Moonshot AI (Feb 2025), [arXiv:2502.13189](#). Used in Kimi's long-context models.

QSA – Qwen Sparse Attention

QSA combines block selection with a learned indexer. It forms an index key for each micro-block, selects blocks, and attends to the original, uncompressed K/V inside them. The current causal block is always included.

This separates the **routing representation** from the **retrieved representation**. A block summary helps choose where to look; it does not replace the values read by the main attention operation.

In the repository's configuration, four-token micro-blocks make the index sequence shorter. The learned indexer still needs supervision, and the dense-then-masked implementation still needs a sparse execution path before any speed claim is justified. Check a query in the middle of a block: future members of that same block must remain invisible.

Code entry point: [nano/attention_zoo.py](#). **Paper:** *On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability* (Alibaba, Aug 2026), [arXiv 2608.30320](#). See also the Qwen3.8-Flash-Next model card and SGLang day-0 write-up.

CSA / HCA – Compressed Attention

Our sparse selectors read fewer original entries, but those entries still exist. To reduce their number, we must change the representation of history itself.

CSA and HCA build pooled entries from groups of tokens. Learned per-channel weights decide how the group contributes. CSA combines relatively fine pooling with selection; HCA pools more aggressively and reads the closed pooled entries densely. A recent-token branch preserves local detail.

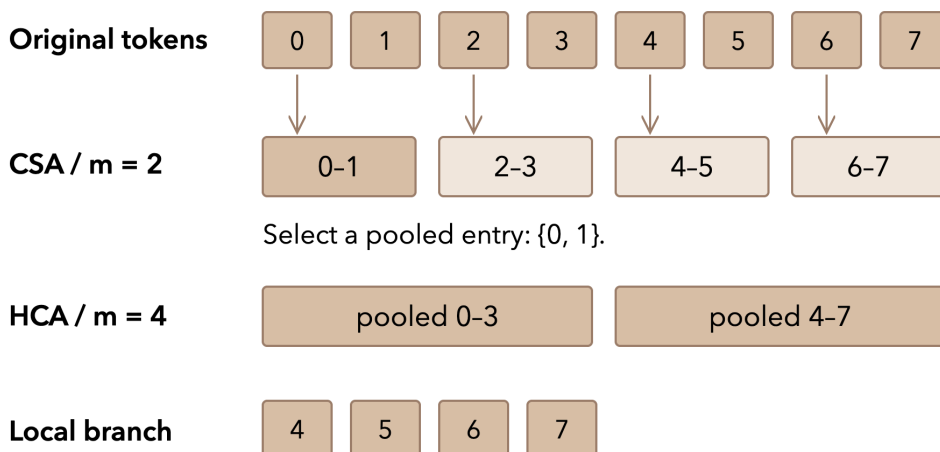
Look for the irreversible step. Once a group becomes one entry, its members are no longer individually addressable through that entry. Two nearby names may compete for the same compressed representation. A smaller number of entries buys a different retrieval problem.

The figure uses groups of two and four so the change is visible. Real group sizes are configuration choices. An entry summarizing positions 0–3 becomes available at position 3, never at position 1: otherwise the summary carries future information backward.

Code entry point: [nano/attention_zoo.py](#). **Paper:** DeepSeek-V4 (DeepSeek, 2026), [arXiv:2606.19348](#).

Compression changes the unit of memory

Pooled history is paired with a recent, uncompressed window.



Select a pooled entry: {0, 1}.

Query = 7. Both pooled groups are closed. Group sizes are illustrative.

Trace it. At query 7, the group 4-7 is complete and can be used. The local branch still has separate keys/values for 4, 5, 6, and 7; the pooled entry is a different representation.

Keep in mind. An entry cannot become visible before every token it summarizes exists. The closure diagram in the failure-modes chapter shows the resulting future leak.

Repository reference: [nano/attention_zoo.py](#)

When the right history still gives the wrong signal

We have changed storage and access. Now hold both fixed. Even when the relevant entries are available, the attention output can be poorly scaled, diluted, or unnecessary. The next tools modify the signal rather than the history budget.

Gated Attention

A head may have nothing useful to contribute at a particular token. Softmax still distributes probability over its available entries. An output gate gives the model another choice: reduce the retrieved contribution before the output projection. Turn the sigmoid gate toward zero and inspect the signal immediately before projection. It should shrink even though the cache and visibility mask remain unchanged. This is output control, not a cache-compression technique. Whether it reduces attention sinks or improves training is an empirical question.

Code entry point: [nano/attention_zoo.py](#). **Paper:** Qwen3-Next / Qwen3.5 (Alibaba, 2025-26).

Attention sinks

A different way to suppress retrieval is to give the softmax denominator an extra destination. Here, each head adds a learned sink logit. Its probability is discarded before multiplication by the ordinary values, so that probability contributes no retrieved value.

Think of the sink as a zero-value option. Raising its logit reduces the total probability assigned to real entries. This is related to, but distinct from, the observed tendency to concentrate attention on early tokens. It does not require evicting or compressing any cached token.

Code entry point: [nano/attention_zoo.py](#). **Paper:** gpt-oss (OpenAI, 2025); *Efficient Streaming Language Models with Attention Sinks* (MIT, 2023) for the diagnosis.

Logit softcapping

A head can also become too confident. Softcapping transforms a score with $c * \tanh(\text{score} / c)$, smoothly limiting its magnitude. Near zero it is approximately the identity.

Catch the bug before running it. A causal mask uses negative infinity for forbidden keys. Passing that mask through $\tanh$ would turn it into a finite negative number, allowing nonzero attention probability. Cap the content scores first, then apply the mask.

The attention score cap and the final vocabulary-logit cap act at different locations. Neither is a guarantee of good calibration or stable training; both change the gradients as well as the values.

Code entry point: `softcap` in [nano/attention_zoo.py](#) for attention logits; `--logit-softcap` in [nano/models/qwen_next_nano.py](#) and [gemma_nano.py](#) for the LM head.

Paper: Gemma 2 (Google DeepMind, 2024), [arXiv:2408.00118](#).

Scalable Softmax

Add more distractor keys while keeping one relevant key's score advantage fixed. Its probability falls because the denominator grows. In a toy row with relevant logit 2 and all other logits 0, its probability is $\exp(2) / (\exp(2) + n - 1)$.

Scalable Softmax makes the logit scale depend on visible sequence length, with a learned per-head factor. That can counter dilution, but it cannot guarantee retrieval for arbitrary score distributions.

Use the same absolute query position in full and cached forwards. A scale based only on the current chunk length would treat one-token decoding as though it were always the beginning of the sequence.

Code entry point: [nano/attention_zoo.py](#). **Paper:** *Scalable-Softmax Is Superior for Attention* (Nakanishi, 2025), [arXiv:2501.19399](#).

Differential Attention

Instead of sharpening one map, compute two and subtract a learned multiple of the second. Shared attention patterns can cancel; differences can remain. The result is no longer an ordinary probability distribution and can contain negative weights.

That signed mixture is the point of the design, but it also makes output normalization important. Setting the subtraction coefficient to zero should recover the implementation's first branch before any additional normalization. It does not automatically make the complete module identical to every conventional attention block.

For our design decision, ask whether the failure comes from unavailable information or unwanted retrieved signal. Subtraction cannot recover a token that the visibility mask excluded.

Code entry point: `nano/attention_zoo.py`. **Paper:** *Differential Transformer* (Microsoft, 2024), [arXiv:2410.05258](https://arxiv.org/abs/2410.05258).

Try it: predict the cache, then inspect it

First use the standard-library calculator. Its default is the four-layer, two-byte model used in this chapter:

```
python book/labs/beginner_workbook.py --lesson cache
python book/labs/beginner_workbook.py --lesson cache --tokens 256
```

Try `--kv-heads 1` or `--window 16`. Predict which counts should change and which should remain the same. The calculator reports logical payloads; it does not allocate a model or estimate runtime.

For a check against actual repository tensors, run the PyTorch companion lab from the repository root in the environment used for the models:

```
python book/labs/attention_cache.py
```

The script runs on CPU with random inputs and no downloads. It instantiates four query heads of width 16, uses float32, and compares one MHA layer with a two-KV-head GQA layer. It prints retained tensor payloads after 128 and 256 tokens and checks cached decoding against a full forward within each module. It does not compare independently initialized modules for output equality.

Before running, write down the four expected byte counts. The float32 cache is twice the two-byte storage used in our opening calculation. For one layer, expect MHA to retain 65,536 then 131,072 bytes; GQA should retain 32,768 then 65,536 bytes. These are tensor payloads, not process RSS or peak training memory.

Change only `n_kv_groups` from two to one. Predict the result again. If the byte count changes but the output shape does not, you have observed the separation between stored KV heads and query heads directly.

Make the design decision

Your observed bottleneck	First mechanism to investigate	Evidence still needed
Too many bytes per cached token	GQA, MLA, K-as-V	Actual allocations and trained quality
Old context is rarely useful	SWA with a chosen window	Retrieval outside the window
Relevant history is distant and sparse	DSA, MoBA, QSA	Selector recall and sparse-kernel timing
Too many historical entries	CSA / HCA	Detail lost through pooling
Available context produces poor updates	Gates, sinks, score transforms	Signal diagnostics and task evaluation

For the running model, choose one intervention and say what information it shares, discards, or reconstructs. Then name one workload on which you expect it to fail. This is a design hypothesis you can test.

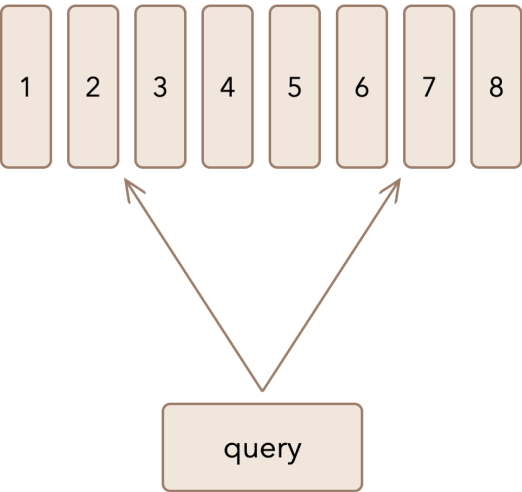
For a deeper implementation exercise, return to the DSA family. The hybrid supports shared indexers (`--index-share`) and hierarchical full/reindex/reuse selection (`--index-modes` with `--candidate-top-k`). Those features reuse routing work or restrict candidate sets; the current dense scoring path does not establish the corresponding runtime saving. The architecture guide records the complete commands and parameter relationships.

We have kept a collection of historical entries, however small or selective. The next chapter asks a more radical question: what if the model folds history into a fixed-size state instead?

Two ways To remember

Keep past entries, or keep an evolving summary.

Addressable history

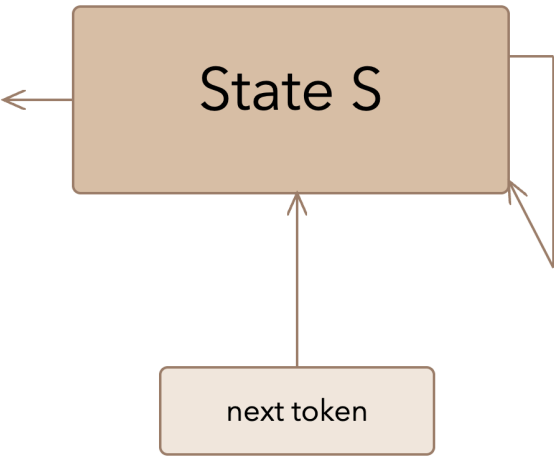


Selection changes access.

Compression changes storage.

Keep separate token entries.

Recurrent state



write / decay / read

History shares one state.

Hybrid: alternate the two



Remember with a state instead of a list

Attention stores addressable entries from earlier positions. A recurrent mixer stores a summary called a **state**, updates it when a token arrives, and reads from it. It resembles taking notes as you read instead of keeping every original page on your desk.

Start with the scalar recurrence $\text{state} = 0.5 * \text{state} + \text{input}$. From state zero and inputs 2, 0, 4, the states are 2, 1, and 4.5. The last state contains a decayed contribution from all three inputs: $0.25*2 + 0.5*0 + 4$. It does not preserve a separately addressable copy of each input.

The multiplier 0.5 is **decay**. A learned **gate** can make such a multiplier depend on the current input. A matrix state stores multiple associations rather than one scalar. The architectures below differ mainly in what they decay, what they write, and how they read.

Gated DeltaNet

Imagine a key asking for a value already represented in memory. Writing another copy on top can reinforce a mistake. A delta-rule update first predicts the stored value at that key, subtracts it from the desired value, and writes a correction.

In the repository's key-by-value orientation, first decay S , then compute $\text{prediction} = K.T @ S$. The error is $\text{value} - \text{prediction}$. Write the outer product of the key and the scaled error into S . An **outer product** forms a matrix: $[1, 0]$ outer $[1, 0.5]$ is $\begin{bmatrix} 1 & 0.5 \\ 0 & 0 \end{bmatrix}$.

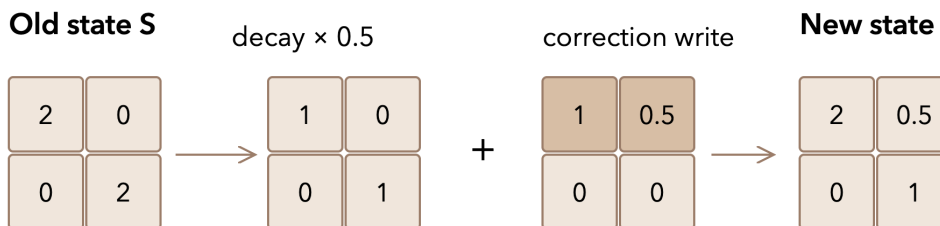
The worked diagram begins with $S = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$. Decay by 0.5 gives the identity matrix. Key $[1, 0]$ reads $[1, 0]$; desired value $[3, 1]$ leaves error $[2, 1]$. A correction rate of 0.5 gives a new state $\begin{bmatrix} 2 & 0.5 \\ 0 & 1 \end{bmatrix}$. Reading it with the same key yields $[2, 0.5]$: we moved halfway toward the desired value, rather than overwriting everything.

This isolates the recurrence. The full module adds projections, normalization, query scaling, and output gates. Those operations matter when comparing actual model outputs.

Code entry point: [nano/attention_zoo.py](#). **Paper:** *Gated Delta Networks* (NVIDIA/MIT, 2024); shipped in Qwen3-Next (2025).

One worked example: correct an association

One head; a two-by-two state, with explicit illustrative inputs.



$$k = [1, 0], v = [3, 1], \beta = [0.5, 0.5]$$

Old prediction after decay = $k^T S = [1, 0]$.

Correction = $\beta \odot (v - \text{prediction}) = [1, 0.5]$.

Read with $q = [1, 0] \rightarrow [2, 0.5]$. Output gates and Q scaling omitted.

After decay, the delta rule writes the discrepancy between the desired value and the value already stored at the key. This diagram uses the source's key-by-value state orientation.

Worked example. Decay $\text{diag}(2, 2)$ by 0.5. Key $[1, 0]$ predicts $[1, 0]$; value $[3, 1]$ leaves error $[2, 1]$. With beta $[0.5, 0.5]$, the first row receives $[1, 0.5]$, giving state $\begin{bmatrix} 2 & 0.5 \\ 0 & 1 \end{bmatrix}$.

Where it breaks. This isolated recurrence omits query scaling, output normalization, gating, and projections. It must not be compared directly with a complete model's projected output.

Repository reference: [nano/attention_zoo.py](#)

KDA – Kimi Delta Attention

Gated DeltaNet uses one decay scale per head. KDA gives each key channel its own decay. This is a difference in the resolution of the forgetting decision.

For a state with two key rows, begin at $\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$. A shared decay of 0.5 gives $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. Per-row decays $[0.5, 0.9]$ instead give $\begin{bmatrix} 1 & 0 \\ 0 & 1.8 \end{bmatrix}$ before the correction write. One part can fade faster than another.

A channel is a coordinate, not a guaranteed semantic category. Training must learn a useful division. In the source, inspect the decay tensor's shape and which state axis it multiplies.

Code entry point: [nano/attention_zoo.py](#). **Paper:** Kimi Linear (Moonshot AI, 2025).

Lightning Attention

Our opening recurrence used a fixed decay. Lightning Attention uses head-dependent fixed decay with an additive associative write, rather than a token-dependent delta correction. Heads can operate at different timescales.

In a scalar example with decay 0.5, key 1, value 2, and old state 4, the new state is $0.5 \cdot 4 + 1 \cdot 2 = 4$. A later zero write produces 2. The decay policy stays fixed even when the token changes.

A fixed recurrence can admit efficient parallel formulations. The educational implementation and a paper's optimized kernel need not have the same speed. Count arithmetic and time the actual implementation separately.

Code entry point: [nano/attention_zoo.py](#). **Paper:** TransNormerLLM (OpenNLPLab, 2023), [arXiv:2307.14995](#); Lightning Attention-2 (2024), [arXiv:2401.04658](#); shipped in MiniMax-01 (2025) and the Ling 2.5/2.6 family (2026; the *Ling and Ring 2.6* report is [arXiv:2606.15079](#)).

Mamba-2 and Mamba-3

A state-space model also updates state over time. In the Mamba-2-style recurrence here, a token controls a step size Δ . With learned negative A, decay is $\exp(\Delta \cdot A)$. Larger steps decay old state more strongly. Input-dependent vectors B and C control writing and reading.

For a one-dimensional example, let $A = -1$, $\Delta = \log(2)$, old state = 2, input $x = 1$, and $B = C = 1$. Decay is 0.5. The simplified update gives state = $0.5 \cdot 2 + \log(2) \cdot 1 \cdot 1 = 1.6931$; reading with C gives 1.6931. This omits the full output pathway and convolution.

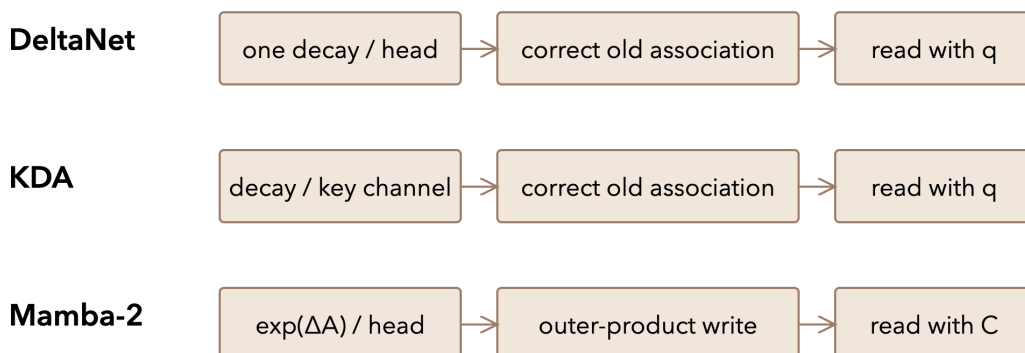
Mamba-3 adds a richer discretization and, when enabled here, phase rotation. A **trapezoidal** write uses information from adjacent time steps; phase gives another way for state to evolve instead of only fading. You can first use Mamba-2, then inspect the extra previous-input and phase state that those flags require.

For fixed dimensions, recurrent state size stays bounded as the sequence grows. Full-history attention grows its cache; a sliding window can also bound storage, but discards direct access outside its window. Bounded decode state does not bound all training activations.

Code entry point: [nano/attention_zoo.py](#). **Papers:** *Transformers are SSMs* / Mamba-2 (Dao & Gu, 2024), [arXiv:2405.21060](#); Mamba-3 (Mar 2026), [arXiv:2603.15569](#). Nemotron 3 (NVIDIA) ships Mamba-2.

Forget, write, read

These are different recurrent updates, not interchangeable names.



State is carried forward. Output gates, projections, and extra history are omitted.

DeltaNet and KDA use a delta-rule correction. KDA changes the granularity of decay. The Mamba-2 row shows its selective additive write before the output pathway.

Worked example. A two-by-two DeltaNet state stores four scalars per head regardless of whether five or fifty tokens were processed. Mamba's state is head width $\times$ state dimension; the dimensions need not match.

Where it breaks. Bounded recurrent state does not mean bounded training activation memory. Convolution history, trapezoidal input history, and complex phase are additional state when enabled.

Repository reference: [nano/attention_zoo.py](#)

ShortConv

A compact long-term state can lose very recent detail. A short causal convolution adds a small rolling view of recent positions. Each channel has its own learned kernel; causal means it uses the current and earlier inputs only.

Take a three-entry kernel with weights $[1, 2, 1]$ ordered from oldest to newest. On recent values $[2, 3, 4]$, its result is $1 \times 2 + 2 \times 3 + 1 \times 4 = 12$. For the first token, unavailable earlier positions are padded with zeros.

One depthwise convolution has width $\times$ kernel_size weights without bias. Separate width-four Q/K/V convolutions with kernel size three have $3 \times 4 \times 3 = 36$ weights. In cached decoding, preserve the previous kernel_size - 1 inputs per channel.

Code entry point: [nano/attention_zoo.py](#). **Paper:** Kimi Linear (2025), LFM2.5; *Dynamic Short Convolutions Improve Transformers* (2026), [arXiv:2606.03825](#).

Hybrid attention (the 3:1 layout)

A hybrid alternates compact recurrent memory with layers that can directly read token history. A nominal 3:1 pattern is three recurrent layers followed by one attention layer.

For eight layers, the simple repeated pattern contains six recurrent layers and two global attention layers. It has two full-history cache owners instead of eight, assuming each attention layer owns its own cache. This is not a claim of fourfold total memory savings: recurrent states, weights, and other buffers still exist.

The companion model can force its final layer to be global, so short configurations can differ from the nominal ratio. Print the actual schedule before calculating the budget. The design question is how much direct retrieval your workload needs.

Code entry point: `build_layer_pattern` and `LINEAR_MIXERS` in `nano/models/qwen_next_nano.py`; the mixers themselves live in `nano/attention_zoo.py`. **Papers:** Qwen3-Next, Qwen3.5 (Alibaba); Kimi Linear (Moonshot); Ling 2.5/2.6 (report [arXiv:2606.15079](#)); Nemotron 3 (NVIDIA), [arXiv:2604.12374](#).

Compare the memory rules

Mechanism	Forgetting	Write	Direct old-token access
Full attention	Entries kept	Append K/V	Yes, within retained context
Sliding window	Evict old entries	Append K/V	Only within window
Lightning	Fixed per-head decay	Add association	No separate entries
Gated DeltaNet	Input-dependent per-head decay	Correct association	No separate entries
KDA	Input-dependent per-channel decay	Correct association	No separate entries
Mamba-2	Selective step-dependent decay	Add selective write	No separate entries

Compute it and change one input

```
python book/labs/beginner_workbook.py --lesson recurrence
```

The lesson prints the scalar recurrence, DeltaNet correction, and scalar Mamba update. Run again with `--correction 1` to change the correction rate from 0.5 to 1: what should key `[1, 0]` read? Answer: `[3, 1]` in this isolated unit-key example. With rate zero, the decayed state receives no correction.

A two-by-two float32 state has 16 bytes of logical payload per head. Ten or a thousand processed tokens do not change that matrix's shape. List any extra convolution or phase buffers before comparing total memory. Next, we will control the scale and position information of the vectors that these mixers receive.

Keep numbers well scaled and positions distinguishable

A vector can have the right shape and still be difficult to train with: its values might become very large, or positions might be hard to distinguish. Normalization acts on scale. Position encoding acts on how order enters the computation. We will calculate both before choosing a variant.

RMSNorm

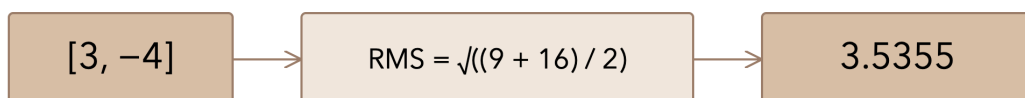
The root-mean-square magnitude of $[3, -4]$ is $\sqrt{(9+16)/2} = \sqrt{12.5}$, about 3.5355. Divide each coordinate by that number to get $[0.8485, -1.1314]$. The normalized vector has RMS one when epsilon is omitted and the learned gain is one. Real code uses $\sqrt{\text{mean}(x*x) + \text{epsilon}}$ to avoid division by zero and multiplies by a learned gain per coordinate. Accumulating squares in float32 helps avoid overflow in lower-precision inputs.

LayerNorm differs: it first subtracts the mean. For $[3, -4]$, the mean is -0.5, so the centered vector is $[3.5, -3.5]$; ignoring epsilon and affine parameters, LayerNorm returns $[1, -1]$. RMSNorm preserves the direction before applying its learned gain, but does not center the values.

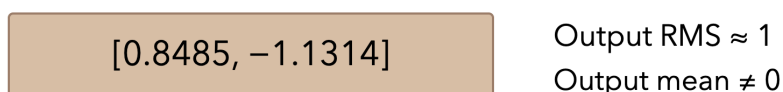
Code entry point: `nano/models/qwen_nano.py`. **Paper:** *Root Mean Square Layer Normalization* (2019).

One worked example: RMSNorm

Two coordinates are enough to separate RMS scaling from centering.



Divide each input coordinate by 3.5355:



Unit learned gain; epsilon omitted for hand calculation. Shape remains [2].

The same scalar RMS divides both coordinates. Signs and direction are preserved, but the mean is not subtracted.

Worked example. For $[3, -4]$, the RMS is $\sqrt{12.5} \approx 3.5355$. Division produces approximately $[0.8485, -1.1314]$, whose RMS is one. The example sets gain to one and omits epsilon.

Where it breaks. Accumulating squared fp16 values can overflow even when the original coordinates are finite. The model implementations use higher precision for the reduction.

Repository reference: [nano/models/qwen_nano.py](#)

Sandwich norm

In a pre-norm block, $x + F(\text{Norm}(x))$, the sublayer F receives normalized input. Its output can still change scale. Sandwich normalization also normalizes that update: $x + \text{Norm}(F(\text{Norm}(x)))$.

If a raw update is $[3, -4]$ or ten times that vector, unit-gain RMS normalization produces the same normalized direction when epsilon is negligible. The residual x is then added outside that normalization.

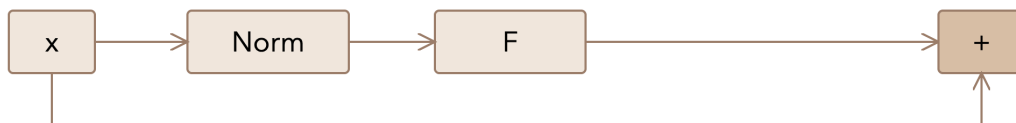
This does not impose a hard bound on the accumulated residual. Learned gains and individual coordinates can still be large. Trace where normalization occurs; adding a normalization after the entire residual sum would be a different block.

Code entry point: HybridBlock with `norm_style="sandwich"` in [nano/models/qwen_next_nano.py](#), and GemmaBlock in [nano/models/gemma_nano.py](#). Not present in [qwen_nano.py](#). **Paper:** Gemma 2 (2024), [arXiv:2408.00118](#); kept in Gemma 3 and 4. OLMo 2 (2024) uses the post-norm half alone.

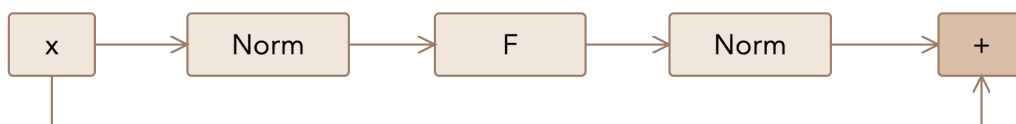
Normalize the input, the update, or both

One sublayer is enough to expose the placement difference.

Pre-norm



Sandwich norm



F is one sublayer. The shortcut carries x without passing through these norms.

Pre-norm produces $x + F(\text{Norm}(x))$. Sandwich norm produces $x + \text{Norm}(F(\text{Norm}(x)))$. The residual shortcut remains outside the sublayer normalizations.

Worked example. Let the raw sublayer update be multiplied by ten. With unit gain and negligible epsilon, output RMS normalization removes that global scale factor; pre-norm alone does not.

Where it breaks. The learned gain and individual coordinates can still be large. Post-normalizing an update is not a hard bound on the accumulated residual stream.

Repository reference: [nano/models/qwen_next_nano.py](#)

QK-norm

Q and K determine attention scores. Normalizing them before the dot product controls their scale at that point, rather than normalizing the attention output.

Take $q = [3, 4]$ and $k = [3, 4]$. Their dot product is 25. Unit-gain RMS-normalizing each vector divides both by $\sqrt{12.5}$, making the dot product 2 before attention's head-width scaling. Learned gains can change this value.

Compare placement: RMSNorm on the residual, QK-norm inside attention, and sandwich norm on the sublayer update control different quantities.

Code entry point: [nano/models/qwen_nano.py](#). **Paper:** *Scaling Vision Transformers to 22B* (Google, 2023); Qwen3.

RoPE

A two-coordinate rotation changes direction while preserving vector length. For a pair (a, b) and angle θ , the new pair is $(a \cos(\theta) - b \sin(\theta), a \sin(\theta) + b \cos(\theta))$. At 90 degrees, $[1, 0]$ becomes $[0, 1]$. At 180 degrees it becomes $[-1, 0]$. RoPE assigns angles using token position and a frequency for each pair, and applies the rotations to Q and K. For fixed content vectors, the positional part of their dot product depends on the difference in positions.

Different coordinate-pair layouts exist in implementations. The repository's split-half pairing need not match adjacent coordinates in a hand-drawn vector. Cached decode must use the correct absolute position rather than restarting rotations at zero.

Code entry point: [nano/models/qwen_nano.py](#). **Paper:** *RoFormer: Enhanced Transformer with Rotary Position Embedding* (2021), [arXiv:2104.09864](#).

NoPE

NoPE removes an explicit position encoding from the relevant layers. It does not remove all order information: causal visibility and recurrent state updates can still distinguish earlier from later inputs.

The experiment is about which positional mechanism is useful. Removing RoPE is not evidence that a model will generalize to longer sequences. Compare order-sensitive tasks and cached offsets, rather than treating the absence of a position table as a guarantee.

Code entry point: the `pos_enc` branch of `GroupedQueryAttention` in [nano/models/qwen_nano.py](#); the `--posenc nope` flag is on [nano/models/qwen_next_nano.py](#).

Paper: *Transformer Language Models without Positional Encodings Still Learn Positional Information* (2022); *The Impact of Positional Encoding on Length Generalization* (2023).

p-RoPE – partial RoPE

Partial RoPE rotates only some coordinate pairs. The remaining pairs stay unchanged. In the toy diagram, the first pair rotates by 90 degrees and the second pair either rotates by 45 degrees or remains fixed.

If a head has eight coordinates, it has four pairs. Rotating half the pairs rotates four coordinates. Rotating none should reproduce the no-rotation calculation; rotating all should reproduce full RoPE at the same frequencies.

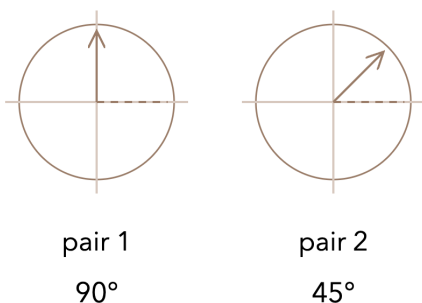
In this repository, identity `cos/sin` entries leave selected pairs unchanged. The actual fraction and frequency schedule belong to the configuration. Keeping some content channels unrotated changes the representation, not the number of cache entries.

Code entry point: `build_rope_tables` in `nano/models/qwen_next_nano.py` (`--posenc rope`), and the global-layer tables in `nano/models/gemma_nano.py`. The rotation itself is `apply_rope` in `qwen_nano.py`. **Paper:** *Round and Round We Go! What makes Rotary Positional Encodings useful?* (Barbero et al., 2024), [arXiv:2410.06205](https://arxiv.org/abs/2410.06205); shipped in Gemma 4 (2026).

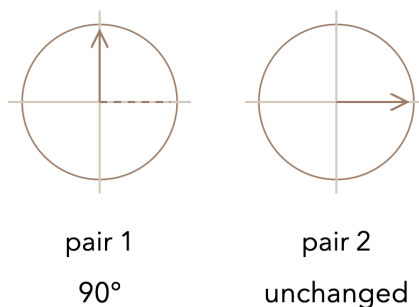
Position lives in coordinate pairs

Dashed vector: original direction. Arrow: transformed direction.

Full RoPE



Partial RoPE



Toy angles; real angles depend on position and the frequency assigned to each pair.

The toy full-rotation panel rotates both coordinate pairs. The partial panel rotates only the first pair and leaves the second pair as content channels.

Worked example. For an original pair $(1, 0)$, a 90-degree rotation gives $(0, 1)$. An unchanged pair stays $(1, 0)$. The actual frequency schedule determines each pair's angle.

Where it breaks. Partial RoPE must use the same rotated dimensions and absolute positions during prefill and decode. Changing the rotary base is not proof of length generalization.

Repository reference: `nano/models/qwen_next_nano.py`

Choose the intervention point

Start with the number you want to control. Large residual magnitudes suggest inspecting sublayer inputs and outputs. Large attention scores suggest inspecting Q and K. Problems distinguishing token order call for position experiments rather than another scale normalization.

Record measurements before and after changing one setting. A smaller activation magnitude is an intermediate observation, not evidence by itself of better language modeling.

Compute and compare

```
python book/labs/beginner_workbook.py --lesson normalization
```

The lesson prints RMSNorm, LayerNorm, a Q/K dot-product comparison, and a 90-degree rotation.

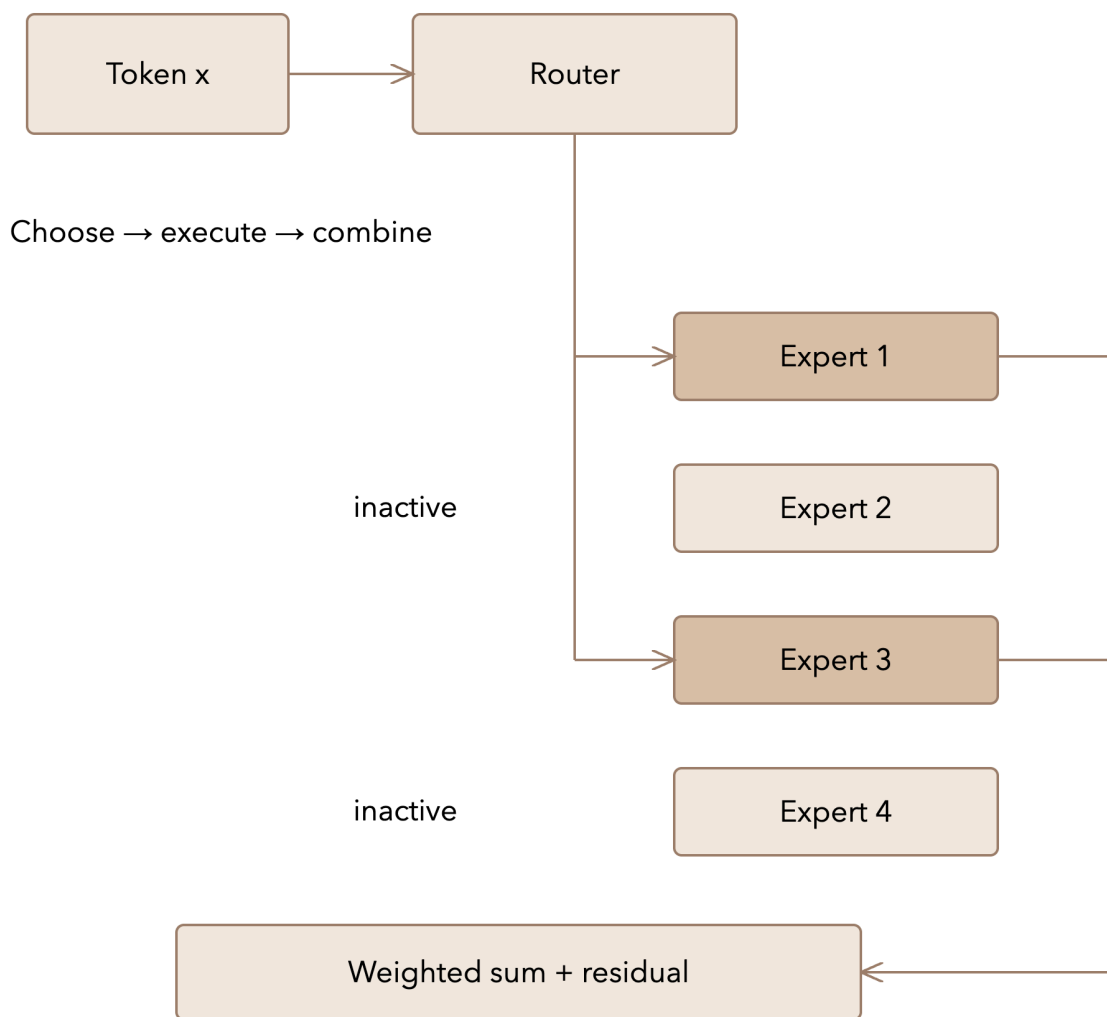
Choice	Acts on	What stays the same
RMSNorm	Vector magnitude	Vector shape
LayerNorm	Mean and variance	Vector shape
QK-norm	Query/key magnitudes	Number of history entries
Sandwich norm	Sublayer input and update	Residual shortcut
RoPE	Query/key coordinate pairs	Pair lengths
Partial RoPE	Only selected pairs	Unrotated coordinates

Exercise. RMS-normalize $[0, 2]$ with unit gain and no epsilon. Answer: $[0, \sqrt{2}]$, not $[0, 1]$. RMS normalization is different from dividing by Euclidean length. Now explain why an eight-coordinate head can rotate two pairs without changing its cache shape.

Capacity is not Active compute

Experts store capacity; routing allocates work.

Top-2 token choice / four experts



Source: nano/models/deepseek_nano.py; illustrative k=2

Ayoub Kirouane / nano-models

Choose which feed-forward network does the work

In our decoder, every token passed through the same FFN. A mixture of experts, or MoE, stores several FFNs and chooses a few for each token. A **router** produces scores used for that choice. An **expert** is a learned network, not a manually labelled subject specialist.

We will use four experts and activate two per token. Separate three counts: stored expert parameters, active expert parameters per token, and how many tokens each expert receives. They answer different questions.

MoE – Mixture of Experts

Suppose a token selects experts 0 and 2 with weights 0.75 and 0.25. If their outputs are $[2, 0]$ and $[0, 4]$, the combined output is $[1.5, 1]$. The router chooses both the path and the mixture weights.

Each bias-free gated expert of width $d = 4$ and hidden width $h = 8$ contains $3 \cdot d \cdot h = 96$ weights. Four experts store 384 expert weights; two selected experts use 192 per token. A width-four router over four experts adds another 16 weights, and actual work also includes selection and dispatch.

This is why total parameters and active parameters are not interchangeable. A larger expert pool can increase stored capacity without running every expert, but training quality and hardware efficiency still need measurement.

Code entry point: `nano/models/deepseek_nano.py`. **Papers:** *Switch Transformer* (Google, 2021); DeepSeek-V3 (2024).

Different tokens take different routes

Illustrative top-two routing across four experts.

Token → expert weights

	E1	E2	E3	E4
token 1	0.7	–	0.3	–
token 2	–	0.4	–	0.6
token 3	0.2	–	0.8	–
Assignments / expert	2	1	2	1

Each row sums to one. Six assignments do not imply equal expert load.

Each token activates two experts and combines their outputs using the shown weights. Expert counts are computed from the displayed assignments.

Worked example. Three tokens with top-k = 2 create six token-expert assignments. Their loads are [2, 1, 2, 1], even though every token activates the same number of experts.

Where it breaks. Balanced work is not guaranteed by top-k. A shared expert, if present, adds work for every token beyond these routed assignments.

Repository reference: [nano/models/deepseek_nano.py](#)

Sigmoid routing

Softmax makes affinities compete across all experts before selection. Sigmoid scores each logit separately, then the selected affinities can be normalized to sum to one.

Use logits $[\log(3), 0, -\log(3)]$. Sigmoid affinities are $[0.75, 0.5, 0.25]$. Select the first two and renormalize: $[0.6, 0.4]$. Softmax on the same logits followed by top-two renormalization gives $[0.75, 0.25]$. The selected experts agree here, but their combine weights differ.

Apply the nonlinearity at the correct stage. Renormalizing every expert before a later top-k is not always equivalent to renormalizing only the selected affinities.

Code entry point: [nano/models/deepseek_nano.py](#). **Paper:** DeepSeek-V3 (2024), [arXiv:2412.19437](#).

DeepSeek-V4 routing: hash bootstrap, Sqrt(Softplus), clamped SwiGLU

These are three distinct switches in the companion model. Learn each separately before enabling them together.

A hash bootstrap uses token identity to choose fixed expert assignments in selected early layers. The expert weights and combine weights can still learn. Fixed assignment does not guarantee balanced traffic when token frequencies are uneven.

Sqrt(Softplus) changes the affinity function to $\sqrt{\log(1+\exp(\text{score}))}$. Unlike sigmoid it is not bounded by one, although selected weights are subsequently normalized. Numerical implementations must evaluate softplus stably.

Clamping limits gate/up values inside expert SwiGLU before their product. That targets activation extremes; it does not change the expert count. The source flags expose these choices independently, making one-factor experiments possible.

Code entry point: [nano/models/deepseek_nano.py](#). **Source:** DeepSeek-V4 (2026), [arXiv:2606.19348](#) §2.1; shipped in V4-Flash (284B/13B) and V4-Pro.

Expert-choice routing

Token choice asks each token for its preferred experts. Expert choice asks each expert for its preferred tokens. If four tokens each use two experts, there are eight assignments. Four experts with capacity two also supply eight slots, but individual tokens may receive different numbers of slots.

Balanced expert counts do not ensure every token is served. More importantly, ranking across an entire sequence can make an earlier token's assignment depend on future tokens. That violates causal decoding. Treat this implementation as a contrast and a diagnostic, not an interchangeable autoregressive router.

Code entry point: [nano/models/deepseek_nano.py](#). **Paper:** *Mixture-of-Experts with Expert Choice Routing* (Google, 2022), [arXiv:2202.09368](#).

Shared expert

A shared expert processes every token in addition to the selected routed experts. If top-k is two and there is one same-size shared expert, each token runs three expert networks rather than two.

The design gives all tokens a common path while the routed pool can learn different behavior. Whether the paths actually specialize is something to measure. Include the shared expert in active-work and parameter counts.

Code entry point: [nano/models/deepseek_nano.py](#). **Paper:** DeepSeek-V2 / V3.

Aux-loss-free load balancing

An auxiliary loss is an extra training objective. A balancing loss can encourage even expert usage, but its gradients also interact with the next-token objective.

The alternative here maintains an expert-selection bias and adjusts it from usage statistics. Underused experts become easier to select. The bias affects selection, while the original affinities determine output combination weights.

This avoids adding that balancing penalty to the gradient objective. It does not mean all gradients or routing outcomes are unaffected: changing which experts run changes the learning path. Log both assignment counts and biases.

Code entry point: [nano/models/deepseek_nano.py](#). **Paper:** DeepSeek-V3 (2024).

LatentMoE

Instead of running experts at residual width d , first project to a smaller width r , run the experts there, and project back. The intermediate space is called a latent space.

With $d = 4$, $r = 2$, hidden width $h = 8$, and four gated experts, expert weights fall from $4 \times 3 \times 4 \times 8 = 384$ to $4 \times 3 \times 2 \times 8 = 192$. Two shared bias-free projections add $2 \times d \times r = 16$, for 208 weights before the router and other parameters.

That saving comes with a bottleneck and projection work. It does not establish better accuracy or lower latency. Keep expert hidden width and router details explicit when doing this comparison.

Code entry point: [nano/models/deepseek_nano.py](#). **Paper:** Nemotron 3 Super (NVIDIA, 2026), [arXiv:2604.12374](#).

Compute the route

```
python book/labs/beginner_workbook.py --lesson experts
```

The lesson ranks expert scores, prints selected expert IDs, normalizes their weights, and computes the mixture and parameter counts above. Try `--router-logits -1 0 1`: the winners must become IDs 2 and 1, not the first two entries. Equal scores prefer the lower ID in this toy implementation; other routers can use different tie rules. It also counts loads for three tokens with routes $\{0,2\}$, $\{1,3\}$, and $\{0,2\}$: expert loads are $[2,1,2,1]$.

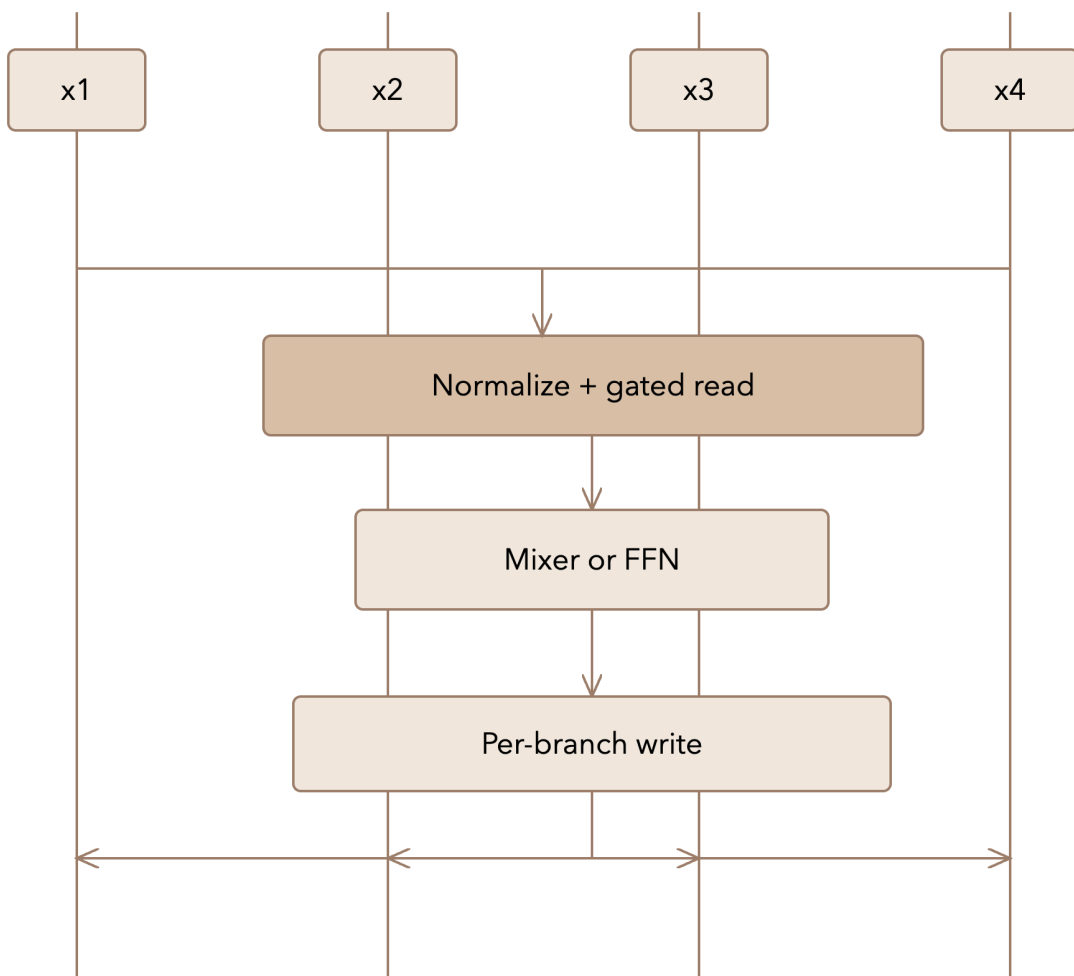
Design	Main choice	What to count
Dense FFN	Same network for all tokens	One full FFN per token
Token-choice MoE	Experts per token	Total weights, active weights, loads
Expert choice	Tokens per expert	Capacity and tokens left unserved
Shared expert	Always-active extra path	Additional work for every token
LatentMoE	Smaller expert representation	Expert savings plus projections

Exercise. Five tokens choosing two experts create how many assignments? Answer: ten. Does that imply equal work across four experts? No. Next we will change the paths between blocks, rather than which FFN processes a token.

Widen the Information path

Gated Residual keeps multiple streams through depth.

Four branches / one sublayer



Branches persist; gates control read and write.

Source: nano/components.py:GatedResidual; conceptual

Ayoub Kirouane / nano-models

Change the paths: memory, residuals, and repeated depth

So far we changed attention, recurrence, or the FFN. We can also change how blocks connect. Keep the simple residual rule $\text{new} = \text{old} + \text{update}$ in mind. The first two mechanisms widen that pathway; later ones add lookup memory, share history, select depth, or add prediction targets.

mHC – Manifold-Constrained Hyper-Connections

Instead of one residual vector, keep several streams. A read operation combines them for a sublayer; a write distributes its output back. mHC also transports old streams through a mixing matrix.

For two scalar streams $[2, 6]$, matrix $[[0.75, 0.25], [0.25, 0.75]]$ gives $[3, 5]$ before the new write. Every row and column sums to one. Such a matrix is **doubly stochastic**: it preserves the sum across streams, but does not preserve each stream individually.

Sinkhorn normalization alternates row and column normalization to approximate these constraints. A finite number of iterations need not satisfy both sets exactly. Check row/column sums and whether streams stay distinguishable. The wider pathway also stores more activations.

Code entry point: [nano/components.py](https://github.com/ByteDance/DeepSeek-V3/blob/main/nano/components.py). **Papers:** *Hyper-Connections* (ByteDance, ICLR 2025), [arXiv:2409.19606](https://arxiv.org/abs/2409.19606); *mHC* (DeepSeek, Dec 2025), [arXiv:2512.24880](https://arxiv.org/abs/2512.24880); *mHC-lite* (2026), [arXiv:2601.05732](https://arxiv.org/abs/2601.05732).

GR – Gated Residual

GR retains multiple residual branches but keeps their old values directly instead of mixing them through a learned transport matrix. Learned gates control what the sublayer reads and how its output is written to each branch.

In a stripped-down write example, old branches $[2, 6]$, scalar sublayer output 4, and write weights $[0.5, 1.5]$ give $[4, 12]$. The weights need not sum to one. The actual read includes branch-wise RMS normalization and elementwise gates; the scalar example isolates only the write.

Compare with the preceding mHC calculation: mixing old information and distributing a new update are separate operations. A diagram that shows only the final addition can conceal this distinction.

Code entry point: [nano/components.py](#). **Paper:** *On the Design of Qwen3.8-Next Architecture* (Alibaba, Aug 2026), [arXiv:2608.30320](#).

Read and write are separate from transport

Two branches shown for clarity; matrices are illustrative.

mHC: mix the residual streams



Residual transport

0.75	0.25
0.25	0.75

Learned mixing matrix

GR: preserve each branch



Residual transport

1	0
0	1

No cross-branch mixing

mHC transports the old residual through a mixing matrix before adding weighted sublayer output. GR retains each old branch directly and controls the read and write with gates.

Worked example. For old scalar branches [2, 6], the shown mHC matrix produces [3, 5] before the new write. GR's transport leaves [2, 6] unchanged before its gated write.

Where it breaks. A doubly stochastic matrix preserves the sum across streams; it is not the identity on every stream. Finite Sinkhorn iterations only approximately satisfy both sets of constraints.

Repository reference: [nano/components.py](#)

PLE – Per-Layer Embeddings

An ordinary embedding looks up the current token at the input. PLE supplies additional token-specific information at different layers. One row can contain slices for several layers.

With vocabulary 3, four layers, and a two-coordinate slice per layer, the table has $3 \times 4 \times 2 = 24$ values. Looking up token 1 gives eight values, reshaped into four slices. This increases lookup capacity without making every layer a full extra FFN.

The full path may include projections and gates. The repository does not implement a production host-memory prefetch system, so counting table entries is not evidence about offloaded inference latency.

Code entry point: [nano/components.py](#). **Paper:** Gemma 4 (Google, 2026), [arXiv:2607.02770](#).

Engram – conditional memory

PLE's address is the current token. Engram uses recent token patterns. An n-gram is a sequence of n token IDs; a hash maps it to a table row. Multiple hash heads reduce reliance on one collision pattern.

For a purely illustrative hash $(3*a+b) \% 7$, token pair [2,4] maps to row 3. Pair [0,3] also maps to row 3. This is a **collision**: different inputs share an address. This tiny hash explains the concept; the repository uses its own fixed hash scheme.

Retrieved vectors are combined, projected, and gated into the residual. For $n = 3$, decoding must retain the previous two token IDs to build the next suffix. Reset that history between sequences, and test a suffix that crosses the prefill/decode boundary.

Code entry point: [nano/components.py](#). **Paper:** *Conditional Memory via Scalable Lookup* (DeepSeek, Jan 2026), [arXiv:2601.07372](#).

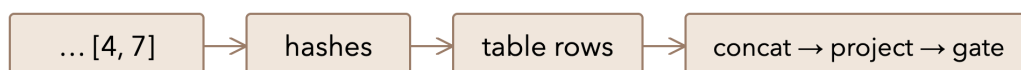
Two different addresses for stored knowledge

Lookup capacity is distinct from dense per-token computation.

PLE: current token + layer index



Engram: suffix ending at the current token



PLE selects a layer-specific slice. Engram combines multiple hashed lookups.

Both add a learned signal to the residual; the lookup key is different.

PLE indexes by token and layer. Engram indexes recent token patterns through multiple fixed hashes, then projects and gates the retrieved vectors.

Worked example. With four layers and PLE width two, one token's table row has eight scalars, reshaped into four two-scalar slices. Engram instead needs the previous token IDs to form the suffix.

Where it breaks. Resetting attention caches alone is insufficient: Engram must reset its n-gram history too. Different suffixes can hash to the same row.

Repository reference: [nano/components.py](#)

Mixture-of-Depths

MoE selects which experts process a token. Mixture-of-Depths selects whether a token receives a block's update. With six tokens and training capacity 0.5, a routed layer selects three updates. The remaining tokens retain their residual values.

Sequence-wide top-k is not causal: future scores can change whether an earlier token is selected. The companion implementation trains a separate predictor and uses its per-token decisions in evaluation. It need not select exactly three at inference.

The educational implementation computes the block and masks updates. Skipped tokens still serve as attention keys. This demonstrates the decision rule but does not realize the compute saving of gathering and processing only selected tokens.

Code entry point: [nano/components.py](#). **Paper:** *Mixture-of-Depths* (Google DeepMind, 2024), [arXiv:2404.02258](#).

Tokens need not receive every layer update

Illustrative train-time capacity = 0.5 on alternating routed layers.

	t0	t1	t2	t3	t4	t5
layer 0	run	run	run	run	run	run
layer 1	run	skip	run	skip	skip	run
layer 2	run	run	run	run	run	run
layer 3	skip	run	skip	run	run	skip

Routed layers: 3 of 6 token updates retained during this training example.

This repo masks updates; it still computes the block and keeps skipped tokens as keys.

Filled cells retain an update. Empty cells leave the residual unchanged. The two dense rows process all six tokens; each routed training row retains three updates.

Worked example. Six tokens at capacity 0.5 select three updates in each routed layer. At evaluation, the causal predictor makes per-token decisions and need not keep exactly three.

Where it breaks. Sequence-wide top-k depends on future tokens. Do not use the training selection rule as if it were a causal per-token inference rule.

Repository reference: [nano/components.py](#)

KV sharing (cross-layer attention)

Two attention layers can ask different questions while reading the same stored keys and values. The later layer computes its own queries and borrows an earlier layer's cache.

If four equally sized layers become two independent cache owners plus two readers, conventional KV payload halves, assuming readers truly alias the owners' storage. Count owners, not the number of Python references to a tensor.

The `--kv-sources` mapping in the companion hybrid specifies earlier owners. A reader must use the correct source and positions. The source configuration also supports more elaborate encoder/reader layouts; begin with one owner and one reader before tracing those recipes.

Code entry point: [nano/models/qwen_next_nano.py](#). **Paper:** Gemma 4 (Google, 2026).

Looped depth – depth-recurrent transformers

Calling the same block repeatedly increases executed depth without storing a separate copy of its weights. If there is one prelude block, one core block repeated three times, and one coda block, effective depth is $1 + 3 \times 1 + 1 = 5$. There are three distinct parameter blocks.

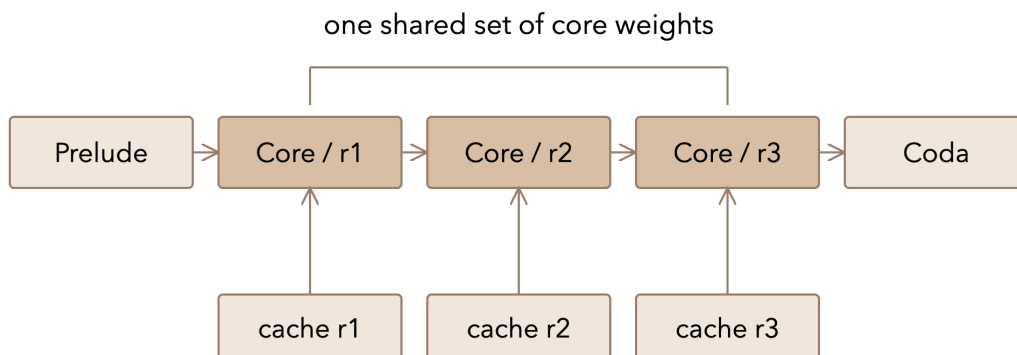
Shared weights do not imply shared generation history. The core's first and second iterations operate at different logical depths, so they need separate cache slots. Increasing loop count increases forward work and cache needs even when parameter count stays fixed.

The companion looped model injects the original representation into repeated updates and can truncate how far gradients propagate through the loop. Truncation changes backward depth, not the number of forward iterations. Its loop count is a configured dial; an adaptive exit policy is not implemented here.

Code entry point: [nano/models/looped_nano.py](#). **Papers:** *Universal Transformers* (Dehghani et al., 2018); Huggins, arXiv [arXiv:2502.05171](#) (Geiping et al., 2025); Ouro, [arXiv:2510.25741](#) (2025).

Shared weights, distinct logical depth

Toy prelude = 1 block, core = 1 block, coda = 1 block, repeats = 3.



Effective depth = $1 + 3 \times 1 + 1 = 5$ blocks.

Parameter blocks = 3. Logical core caches = 3.

The core appears three times in the forward computation but owns one parameter set. Its logical iterations retain different histories during cached generation.

Worked example. Effective depth is $p + r \cdot c + q = 1 + 3 \cdot 1 + 1 = 5$. Only three distinct parameter blocks exist: prelude, core, and coda. The core requires three cache slots; prelude and coda have their own state too.

Where it breaks. Truncating backpropagation does not shorten the executed forward depth. Reusing one core cache across iterations changes the model's history.

Repository reference: [nano/models/looped_nano.py](#)

Parallel block

In a serial block, the FFN sees the attention-updated residual. In a parallel block, both sublayers read the old residual and their updates are added together.

Ignore norms for a scalar example. Let attention return x and the FFN return twice its input. Starting at $x = 1$, serial gives $u=1+1=2$, then $y=2+2*2=6$. Parallel gives $1+1+2*1=4$. Same functions and parameter count, different dependencies.

Independent inputs can enable execution optimizations, but the repository does not automatically run the branches concurrently. Verify dependency semantics before making timing claims.

Code entry point: [nano/models/gpt_nano.py](#). **Papers:** GPT-J (EleutherAI, 2021); PaLM (Google, 2022), [arXiv:2204.02311](#); Falcon (2023).

The Gemma recipe

A model recipe combines choices already introduced: local/global attention, partial RoPE, normalization placement, a gated FFN, embedding tying, and selected cache-sharing rules. These settings interact.

For zero-initialized gain offset w , the gain $1+w$ starts at one. This preserves an initially nonzero normalized signal, unlike a gain initialized directly to zero. For tied embeddings, multiplying input embeddings by $\sqrt{d}$ does not create a second stored embedding matrix.

Trace one configuration through `gemma_nano.py`. Record which layers are local and global, which rotate only some pairs, and which own K/V. Do not apply one layer's cache formula to the entire stack without checking the schedule.

Code entry point: `nano/models/gemma_nano.py`. **Papers:** Gemma 2 (2024), [arXiv:2408.00118](https://arxiv.org/abs/2408.00118); Gemma 3 (2025), [arXiv:2503.19786](https://arxiv.org/abs/2503.19786); Gemma 4 (2026), [arXiv:2607.02770](https://arxiv.org/abs/2607.02770).

MTP – Multi-Token Prediction

The ordinary head predicts token $t+1$ from the representation at t . This repository's additional MTP head combines that representation with the embedding of token $t+1$ and predicts $t+2$.

For the snippet $[A, B, C, D]$, the main valid targets are $[B, C, D]$. The MTP pairs are $h(A) + \text{embedding}(B) \rightarrow C$ and $h(B) + \text{embedding}(C) \rightarrow D$. If you train the first pair to predict B instead, the answer has already been supplied as an input.

This is an extra training objective. Efficient speculative generation would additionally require verification and state rollback, which the repository does not implement. Correct target alignment is the first check, before judging whether the extra objective helps.

Code entry point: `nano/models/qwen_next_nano.py`. **Paper:** *Better & Faster Large Language Models via Multi-token Prediction* (Meta, 2024); DeepSeek-V3; Qwen3.5; Nemotron 3.

Draw the targets before writing the loss

A four-token snippet with no supplied continuation after D.

Input token	A	B	C	D
Main target	B	C	D	–
MTP next embedding	B	C	–	–
MTP target	C	D	–	–

At A: main predicts B; MTP consumes $h(A) + \text{Emb}(B)$ and predicts C.

The main target is one token ahead. The MTP head consumes the next token's embedding and predicts the token after that. A dash means the snippet supplies no valid target.

Worked example. For inputs A, B, C, D, valid main targets are B, C, D. Valid MTP targets are C, D when paired with hidden states at A, B and embeddings of B, C.

Where it breaks. Predicting B from $h(A)$ plus $\text{Emb}(B)$ is a shifted-target bug: the supposed target has already been supplied to the MTP input.

Repository reference: [nano/models/qwen_next_nano.py](#)

Combine structures deliberately

Before combining two options, list their inputs, outputs, and retained state. A recurrent mixer, a lookup memory, and a looped core have different state ownership rules even if each returns a tensor of the same shape.

Keep a copied baseline, change one option, and test shapes, causality, cached equivalence, and gradients. A composition is correct only when these interfaces agree.

Compute the differences

```
python book/labs/beginner_workbook.py --lesson structure --loops 3
```

The lesson prints mHC transport, a GR write, lookup addressing, effective depth, serial/parallel results, and aligned MTP targets. Run again with `--loops 5`: effective depth becomes seven while the three parameter blocks remain three.

Change	Reuses or adds	What must stay separate
mHC / GR	Multiple residual branches	Read, transport, and write
PLE / Engram	Lookup tables	Token address versus suffix address
MoD	Selected updates	Training and causal inference decisions
KV sharing	Earlier K/V	Cache owners and query readers
Looped depth	Core weights	Cache per logical iteration
MTP	Additional training target	Input token and prediction horizon

Exercise. Two core blocks repeated four times, one prelude, one coda: how many effective blocks and distinct parameter blocks? Answer: ten effective blocks, four distinct blocks. The core alone needs eight logical cache positions when all its blocks use cached attention.

Ask the component. Then ask the task.

Five checks turn a design claim into inspectable evidence.

01

Shapes

Can the tensors connect?



02

Causality

Can the future change the past?



03

State equivalence

Does decode match a full forward?



04

Gradients

Does each new component learn?



05

Held-out evaluation

Does the change help this workload?

A falling training loss answers none alone.

Debug with a question you can answer

A correct shape does not prove a correct computation. Before looking at a training curve, choose a tiny input for which you can predict what must happen. The cases below come from the companion repository's debugging notes; they are examples of useful tests, not newly measured failure rates.

Start with three small experiments

Hide the future. Use tokens `[0, 1, 2, 3]`, then change only the last token. Earlier logits must stay the same in causal evaluation. Check the routing choices too: a mask cannot undo information already leaked by a selector.

Split the input. Evaluate four tokens together, then as a three-token prefill followed by one decode step. With dropout disabled and state reset, the last output should match within numerical tolerance. A mismatch suggests missing history, wrong positions, or wrong cache ownership.

Follow the gradient. Pick a parameter that should learn, inspect its gradient, perform an optimizer step, and check whether it changed. An auxiliary loss is a separate learning objective; returning it from a module is insufficient if the training loop never includes it.

A few terms below: **KL divergence** compares probability distributions; **autocast** automatically chooses precision for some operations; **fp16** and **bf16** are two-byte floating-point formats with different ranges and precision; **RNG** means random-number generator. Reusing a seed is useful, but changing construction order can still assign different random values to corresponding weights.

MTP shift

Predicting $t+1$ instead of $t+2$ trains identically. Assert the exact target alignment on a synthetic sequence; different top-1 predictions alone do not prove correct shifting.

Where it breaks: predicting the supplied token

A falling auxiliary loss does not establish correct target alignment.

Given snippet: $A \rightarrow B \rightarrow C$



Wrong: the target was already supplied as the next-token embedding.



Correct: the MTP target is two positions ahead of the original hidden state.

Both rows have the same MTP inputs. Only the prediction target changes. In the wrong row, the head can learn to reproduce an embedding already present in its input.

Worked example. The native targets at position i contain token $i+1$. The MTP loss must compare against targets at $i+1$, which contain token $i+2$; the final unavailable target is omitted.

Where it breaks. Checking whether the two heads have different top-1 outputs does not prove alignment. Assert the exact target IDs on a synthetic sequence.

Repository reference: [nano/models/qwen_next_nano.py](#)

Sinkhorn axis order

Finite iterations only *approach* the doubly stochastic manifold, so whichever axis is normalised last is the exact one. Row sums conserve signal across streams – ending on columns left rows off by $2e-4$ *after training*, invisible at init.

Hyper-connection symmetry

Identical streams under a doubly stochastic matrix stay identical forever, and receive identical gradients – a saddle point training cannot escape. $n=4$ silently behaves like $n=2$. Streams must start distinguishable.

DSA top-k ties

The indexer's ReLU makes exact-zero scores the common case, and `torch.topk` breaks ties by memory order – which differs between a prefill and a one-token decode. Cached generation silently selected different tokens than uncached.

KL with a zero target

Combining masked log-probabilities of $-\infty$ with zero targets can produce undefined arithmetic in some loss paths. Writing it as cross-entropy drops a constant and sidesteps it.

Aux losses in reported metrics

Folding DSA's indexer objective into the logged loss made it read 26.26 against ~ 10.7 for every other variant. Its actual LM loss was 10.75. Auxiliary losses belong in the gradient, not the metric.

Autocast dtype in MoE routing

Expert output is bf16 under autocast while softmax keeps probabilities in fp32, and `index_add_` rejects mismatched types. Unreachable until a latent projection made the MoE input bf16.

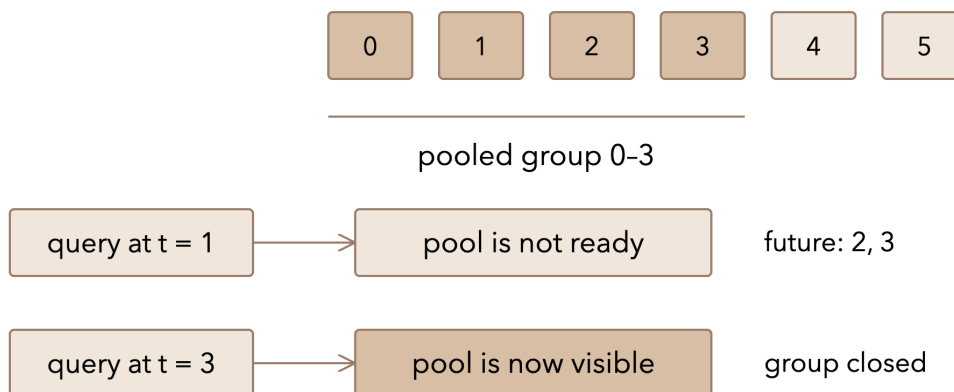
Compressed-entry causality

An entry summarising tokens $[s, s+m)$ may only be read once the group *closes*, at $s+m-1$. Reading earlier leaks the future – and the loss curve looks unusually **good**, not broken.

Where it breaks: an unfinished group

Causal visibility depends on the group's final position.

A pooled entry contains tokens 0-3.



The entry summarizing positions 0-3 may first be read at position 3. Reading it at position 1 leaks information about tokens 2 and 3.

Worked example. Change token 3 while leaving the prefix through token 1 fixed. A correct decoder's output at token 1 is unchanged; an early pooled read can change it.

Where it breaks. A causal mask over pooled entry indices is insufficient unless each entry's availability is mapped back to its original group-closing token position.

Repository reference: [nano/attention_zoo.py](#)

Accelerator built for its device

Every main called `accelerator().device` before the training settings were known. The singleton was created with `mixed_precision="no"`, and the `bf16` and `grad-accumulation` settings `train()` passed afterwards were dropped – every GPU/MPS run logged `Precision: no` and `--grad-accum` changed the log line but not the optimizer. Now `current_device()` reads the device off `PartialState`, and asking the singleton for different settings raises.

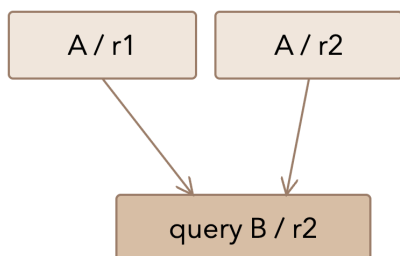
Looped KV cache

A weight-shared core applied r times needs r KV caches. With one, iteration 3 of the current token attends to iteration 1's keys of earlier tokens; prefill and one-token decode then disagree by $\sim 1e-1$ while training is unaffected. `looped_nano` keeps a cache slot per iteration, and its self-check asserts the incremental test *fails* when the slots are shared.

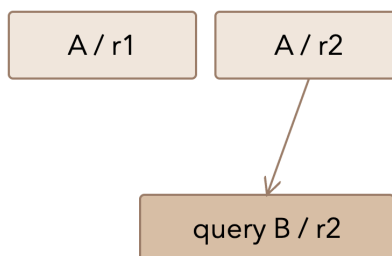
Where it breaks: one cache for every loop

Previous-token history shown; the current-token entry is omitted.

Wrong: mix iterations



Correct: select the matching slot



A and B are token positions; r1 and r2 are depth iterations.

Shared weights do not make the hidden representations interchangeable.

The second iteration for token B should read prior-token keys from its matching logical iteration. A single mixed history also exposes keys from the wrong depth.

Worked example. Uncached and cached forwards can disagree even when ordinary training loss is finite. Compare both execution paths at the same loop count and perturb earlier cache slots to isolate the fault.

Where it breaks. A cache reset between sequences does not fix mixing within a sequence. The core needs a separate cache slot for each logical iteration and core layer.

Repository reference: [nano/models/looped_nano.py](https://github.com/Quintus/nano/blob/main/nano/models/looped_nano.py)

SWA cache trimmed before attending

The sliding-window cache was cut to the window *before* the attention step. A cached prefill longer than the window then gave its early queries nothing but future keys: all-masked rows, softmax of $-\infty$, NaN. Every decode step after the prefill still matched the full forward exactly, so the zoo test passed. It surfaced only when SWA went into the hybrid model, where the NaN prefill output is the next layer's input. The zoo test now compares the prefill output too.

Trimmed cache that still holds the prefill

Sliding-window caches were trimmed with a bare slice, `cache[:, :, -w:]`. The shape said `w` tokens; the storage still held the entire prefill, because a slice keeps its base tensor alive. Every shape and equivalence test passed. Only asserting on `untyped_storage().nbytes()` catches it, and `.clone()` on the slice fixes it. Same fix in

the zoo's SWA, in ShortConv's rolling window (which also grew without bound at kernel 1, because `u[... , -0:]` is the whole tensor), and in `gemma_nano`.

Expert choice is non-causal

Expert-choice routing picks each expert's top tokens over the whole sequence, so which experts process token t depends on tokens after t . A decoder trained with it reads the future through its routing and the loss looks *better*, not broken. The MoE self-check asserts the leak exists, so nobody mistakes the flag for a free balancing fix.

fp32 buffers in a half-precision matmul

`module.to(torch.float16)` casts parameters and buffers, but not tensors *computed* from them. Lightning's decay mask came from an fp32 arange, stayed fp32, and the $(QK^T \odot \text{decay}) @ V$ matmul raised a dtype mismatch. Invisible under autocast, which the training path always uses; only explicit `.to(dtype)` inference hit it. Every zoo entry now passes an fp16/bf16 forward without autocast.

Stale aux loss on a module that did not run

The DSA indexer loss is stashed on each MLA module and collected by a module scan. The MTP block's MLA sets one during training and does not run in an eval forward, so the scan handed back last step's loss and `generate()` received a tuple instead of logits. Every self-check passed; it died on the first mid-training sample. Fix: clear the stashes at the start of every forward, and a check that trains with MTP then evals without it.

Init RNG shifts

Comparing "same model with and without component X" is invalid if X adds modules: it changes how much RNG the weight init consumes, so every weight differs. Detach the component from one model instead.

How a gradient becomes a parameter update

A gradient describes local sensitivity of loss to parameters. An optimizer uses current gradients and sometimes stored statistics to decide the next update. Architecture changes the forward computation; optimization changes how its parameters are learned.

Start with one scalar weight $w = 2$, gradient $g = 0.5$, and learning rate 0.1 . Plain gradient descent produces 1.95 . With squared-error loss $0.5 \cdot (w-1)^2$, the gradient at $w = 2$ is 1 , so that same learning rate instead produces 1.9 . Always specify the loss that generated your gradient.

Momentum keeps a running combination of gradients. **AdamW** also uses per-coordinate squared-gradient statistics and decoupled weight decay. Their state consumes memory in addition to model weights. A learning rate appropriate for one rule is not automatically appropriate for another.

Muon

For a matrix gradient, directions can have very different strengths. Singular values describe those strengths along a matrix's principal input/output directions. You can understand the idea first with a diagonal matrix: positive diagonal entries 10 , 2 , and 0.5 are its singular values.

An exact polar-factor transformation would replace those positive diagonal entries with ones. Muon's finite polynomial iteration approximates such a flattening without computing a full singular-value decomposition. It does not make every direction exactly equal.

The figure uses the repository's five-step polynomial and normalization. For the diagonal example it produces approximately $[0.724, 0.800, 0.755]$ in the original direction order. Those are computed mechanism values, not training improvements.

The optimizer then applies its learning-rate and shape scaling. This implementation uses $\sqrt{\max(1, m/n)}$ for an m -by- n update. Eligible block matrices use Muon; embeddings, normalization gains, biases, and other excluded parameters use AdamW. Parameter-group selection is part of the algorithm, not just an implementation detail.

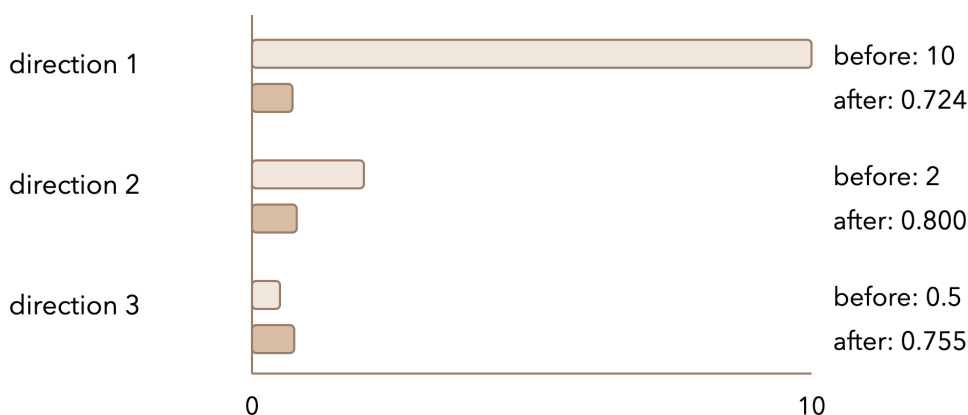
Inspect `nano/optim.py` before changing groups. A two-dimensional normalization gain is not automatically an ordinary matrix projection. The source's synthetic checks test approximate ranges and routing; they do not prove a universal quality or convergence guarantee.

Code entry point: `nano/optim.py`. **Sources:** Keller Jordan's Muon notes and Muon is Scalable for LLM Training, arXiv:2502.16982. MuonClip is a related extension; its QK clipping is not implemented here.

Flattening singular values is approximate

Diagonal toy matrix: $\text{diag}(10, 2, 0.5)$, five Newton-Schulz steps.

Common horizontal scale: singular value 0 to 10



Calculated toy transform; before learning-rate scaling or weight decay.

Bar lengths use one common zero-based scale. The after values are calculated with this repository's polynomial coefficients and normalization, before optimizer learning-rate scaling.

Worked example. The input singular values are 10, 2, and 0.5. Five polynomial steps give approximately 0.724, 0.800, 0.755. An exact polar factor would give one for every nonzero singular value.

Where it breaks. This is a numerical mechanism example, not evidence of improved training or a universal convergence bound. Norm gains and embeddings must remain outside the Muon matrix-update group.

Repository reference: `nano/optim.py`

Compute one learning step

```
python book/labs/beginner_workbook.py --lesson optimizer
python book/labs/beginner_workbook.py --lesson loss
```

The first command computes the scalar update and the diagonal Muon illustration. The second starts with logits $[0, 0, 0]$, takes a gradient step for target index 2, and verifies that this toy loss falls. It updates logits directly to expose the derivative; training a language model instead backpropagates through the head and earlier layers into weights.

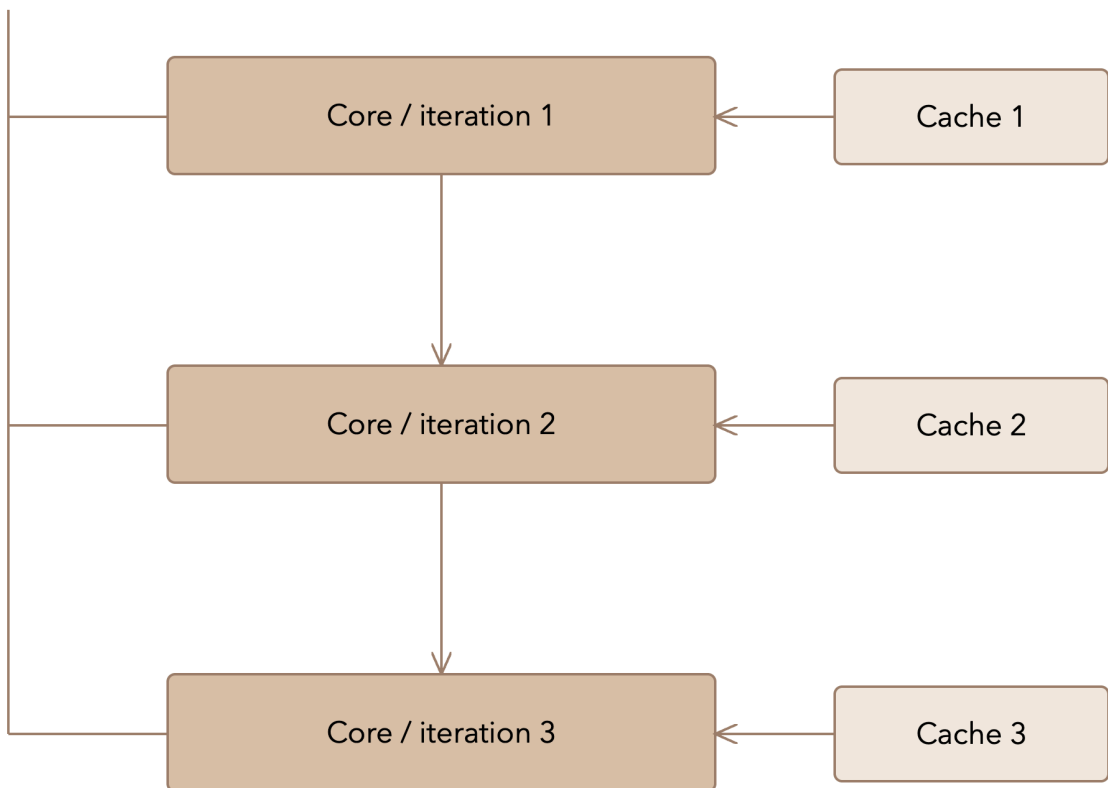
Rule	Simplified idea	Additional state
Gradient descent	Follow the negative gradient	None in the plain rule
Momentum	Combine recent gradients	Running direction
AdamW	Adapt coordinates and decay weights	First/second moments
Muon here	Transform eligible matrix directions	Momentum; AdamW state for other groups

Exercise. With $w = 1.9$ and loss $0.5 \cdot (w-1)^2$, another gradient-descent step of size 0.1 gives 1.81. Calculate both losses: 0.405 and 0.32805. This one-dimensional improvement is not a comparison of optimizers on language modeling. The workflow chapter explains how to set up that comparison.

Reuse the core. Separate the cache.

A depth recurrence spends compute without new weights.

Weights are shared. state is not.



Input injection anchors every iteration.

Effective depth grows with the loop count.

Source: `nano/models/looped_nano.py`; illustrative $r=3$

Ayoub Kirouane / nano-models

Put the pieces together and compare the models

You now know the individual operations. A whole model chooses several at once. That makes a recipe useful for exploration but a poor one-factor experiment: if two recipes differ in attention, FFN, and normalization, a loss difference cannot identify which change caused it.

Begin with the GPT family. Move to the dense Qwen family after you can trace its norms, positions, and KV heads. Then choose one branch: experts for routed capacity, Mamba for recurrent memory, or Looped for shared-weight depth. Qwen-Next is the composition laboratory once those parts are familiar.

Educational family	What to learn first	Comparison to make
GPT	Conventional decoder	Full vs cached execution
Qwen	Modern dense block	Norms, positions, KV heads
DeepSeek	Latent attention and experts	Stored vs active capacity
Qwen-Next	Hybrid and optional components	One option at a time
Looped	Repeated core	Parameters vs executed depth
Gemma	Local/global dense recipe	Cache owners and layer schedule
Mamba	Pure recurrent mixer	State size vs sequence length

These names identify companion implementations and inspirations. They are not promises of production-checkpoint compatibility or equivalent quality.

GPT Nano: the reference decoder

`gpt_nano.py` is the entry point for a conventional decoder with learned positions, LayerNorm, and a GELU MLP. It also makes attention interchangeable through the zoo registry. Start here when comparing a mixer under a familiar block layout. Its parallel-block option changes the dependency between attention and the FFN without automatically scheduling the branches concurrently.

Exercise. Draw the dependency graph for serial and parallel blocks. Hold parameter count fixed and ask whether equal compute is also enough to make their

representational capacity identical. Explain which current-layer information the parallel FFN cannot see.

Qwen Nano: the modern dense baseline

`qwen_nano.py` combines RMSNorm, SwiGLU, rotary position encoding, and GQA with QK normalization. This is a useful reference for experiments that need a dense decoder with modern block conventions. Many other files reuse its primitives or training machinery.

Exercise. List the parameters that change when replacing LayerNorm with RMSNorm and an ordinary MLP with SwiGLU. Separate the effects of normalization, activation, hidden width, and attention head sharing before proposing an ablation.

DeepSeek Nano: latent attention and experts

`deepseek_nano.py` combines a latent attention mechanism with shared and routed experts. Router affinity, balancing, expert-choice demonstrations, hash bootstrap, clamping, and latent expert width live in this family. The model is a compact way to study conditional parameter activation.

Boundary. The attention reconstructs K/V from cached latents and does not reproduce the full decoupled rotary-key design of production MLA. The V4-Flash routing preset belongs here, but it does not add CSA/HCA and mHC to this model. A preset name should never substitute for reading its actual fields.

Qwen-Next Nano: the composition laboratory

`qwen_next_nano.py` combines cheaper sequence mixers with selected attention layers. It is also the integration point for residual variants, explicit memory, depth routing, cache sharing, and multi-token prediction. It offers the broadest set of interactions, so it is the worst place to begin by enabling every flag at once.

Boundary. Its FFN is dense. The Qwen Flash preset demonstrates Gated DeltaNet, QSA, Engram, GR, and MTP; it does not reproduce the production model's sparse-expert FFN. The causal encoder-decoder preset demonstrates cross-layer KV ownership with GQA. The hierarchical MLA indexer is a separate configuration because this repository's MLA cannot share K/V through that interface.

Exercise. Print the actual layer schedule and annotate which modules own cache, which borrow it, and which carry recurrent state. Then enable one structural feature and update the map.

Looped Nano: parameters versus effective depth

`looped_nano.py` divides computation into a prelude, a repeated core, and a coda. If those sections contain p , c , and q blocks and the core repeats r times, the effective block depth is $p + r \cdot c + q$. Parameter depth counts the core once. Runtime and retained generation state generally do not.

Boundary. The loop count is a dial, not a learned per-token exit policy. Truncated backpropagation limits the iterations through which gradients flow; it does not mean the forward pass skipped earlier iterations. Each logical core iteration needs its own cache.

Exercise. Compare a fixed-weight model at two loop counts. Report latency, cache growth, and held-out loss separately. Extra inference computation has no guaranteed benefit without a suitable training distribution.

Gemma Nano: a dense system of small choices

`gemma_nano.py` combines local/global attention scheduling, two rotary bases, partial rotary channels, K-as-V on appropriate layers, sandwich normalization, GeGLU, and a final logit cap. It illustrates how a model family can emerge from coordinated refinements rather than one new mixer.

Exercise. Make a layer-by-layer table of window, position encoding, and K/V ownership. Distinguish the whole-model file from `gemma4_like.json`, which only adds selected Gemma-inspired features to the hybrid family.

Mamba Nano: a pure recurrent baseline

`mamba_nano.py` stacks selective state-space mixers without attention or a separate FFN sublayer. Its generation history is recurrent state, not token K/V. Trapezoidal and complex-state options expose additional state-space ideas at nano scale.

Boundary. Constant retained state means constant in sequence length for a fixed architecture and batch size. It does not mean zero memory, constant training activation memory, or automatic speed superiority. Inspect every carried tensor when enabling a new option.

Read the presets as executable hypotheses

Each preset should answer “which combination am I trying?” rather than “which commercial model have I reproduced?” Defaults and explicit command-line overrides still matter.

Preset	Entry point	What it demonstrates
<code>kimi_like.json</code>	<code>qwen_next_nano</code>	KDA, ShortConv, gated MLA, 3:1 layout
<code>qwen_flash_next.json</code>	<code>qwen_next_nano</code>	GDN/QSA hybrid, Engram, GR, MTP; dense FFN
<code>deepseek_v4_like.json</code>	<code>qwen_next_nano</code>	mHC and MTP in the hybrid
<code>deepseek_v41_ced.json</code>	<code>qwen_next_nano</code>	GQA cache owners and later readers; Engram/MTP
<code>gemma4_like.json</code>	<code>qwen_next_nano</code>	PLE and cross-layer KV sharing
<code>deepseek_v4_flash.json</code>	<code>deepseek_nano</code>	Hash bootstrap, sqrt-softplus routing, expert clamping
<code>latent_moe.json</code>	<code>deepseek_nano</code>	Experts operating in a compressed space
<code>wikitext.json</code>	<code>qwen_next_nano</code>	Dataset configuration plus a KDA hybrid

The [configs directory](#) is the authoritative companion. The complete field values are included in the repository atlas later in this edition.

From a hand calculation to an experiment

The workbook made each mechanism small enough to check. A model experiment adds data, many parameters, an optimizer, and a hardware environment. Keep those choices explicit so you can explain any observed change.

The minimal training loop

```
read input tokens and shifted target tokens
compute logits with a causal forward pass
compute next-token loss over valid target positions
add any required auxiliary training losses
clear old gradients
backpropagate the total objective
update parameters with the optimizer
```

An **epoch** is one pass through the chosen training data. A **step** is one optimizer update. A **checkpoint** saves parameters and usually optimizer state plus enough configuration to resume. **Validation** evaluates held-out data without updating weights; it asks whether learning transfers beyond the examples used for updates.

Run `python book/labs/beginner_workbook.py --lesson loss` to inspect one tiny logit update first. That lesson is not a model-training loop: it isolates the loss derivative. The repository model scripts supply the larger machinery.

Compare one change fairly

Write down the baseline, one architectural change, and the cost you want to improve. Use the same data split and token budget. Record parameter counts, active work, and memory rather than assuming they are matched. Use multiple seeds for a serious quality claim; one lucky initialization is not evidence of a reliable improvement.

If two variants are intended to share initial weights, construct a baseline and copy compatible parameters. Merely setting the same seed before creating two differently structured modules can shift which random draws reach each parameter. First verify the intended equality directly.

The following sections connect those principles to the actual repository. They include proposed experiments; no language-model training or throughput benchmark was run for this beginner rewrite.

Configuration is an experimental record

`nano/config.py` combines defaults, a config file, and explicitly typed flags, in that order. An `argparse` default must not silently replace a value supplied by the file. A resolved snapshot records architecture, training settings, data source, seed, and environment information alongside checkpoints.

```
python -m nano.models.qwen_next_nano \
  --config configs/kimi_like.json
```

This command trains, so inspect the config and compute requirements before running it. To reproduce the resolved configuration, pass the saved `config.resolved.json` as the next run's config. To resume optimizer and model state, use the checkpoint resume path as well: reconstructing settings and resuming training are different operations.

A seed alone is insufficient. Adding a module consumes random numbers during initialization and can change unrelated weights. For a surgical ablation, initialize once, copy the model, and alter the relevant component while preserving compatible weights. For a new architecture whose shapes differ, use multiple seeds and make the mismatch explicit.

Data loading and target alignment

`nano/data.py` supports a bundled corpus, a local file, or a text field in a dataset. At this teaching scale the corpus is joined into a string. It is not a streaming pretraining data pipeline. A production-scale extension would need tokenization to disk, sharding, deterministic sampling, and a separately designed evaluation split.

The native training data supplies already shifted targets. At position t , logits predict the next token. Keep this convention in mind when moving to an adapter that expects unshifted labels. On a synthetic sequence of numbered tokens, write the inputs and targets on paper and compare with the tensors passed to loss. This simple check exposes a family of otherwise quiet off-by-one bugs.

Acceleration and precision

`nano/accel.py` owns the Accelerator setup, device selection, accumulation, and precision settings. Constructing shared accelerator state too early can lock in settings before the training configuration is known. Read the repository's check around conflicting settings before extending the singleton.

Low precision changes more than parameter storage. Reduction arithmetic, routing probabilities, temporary position tensors, and scatter/add destinations can have different dtypes. Autocast and an explicit low-precision model exercise different paths. Test both if both are advertised.

The repository roadmap records CUDA, large-model, and multi-GPU validation as outstanding. Distributed plumbing is present; this book does not turn that into a validated scaling claim. For an auxiliary head, establish that its gradients are computed inside the wrapped forward and synchronized across ranks before relying on a distributed experiment.

The Hugging Face adapter

`nano/hf.py` converts native model calls into the configuration and output conventions expected by a causal-LM interface. `examples/sfttrl.py` shows the intended SFT integration. Read the pinned local example rather than assuming that every trainer or generation feature is supported.

The adapter's main responsibilities are label shifting, auxiliary loss collection, and rejecting unsupported padding. Equal-length token blocks avoid padding. Packing is not automatically a substitute: flattening examples requires correct boundary handling and compatible kernels. The documented example uses `loss_type="nll"` and disables gradient checkpointing.

Internal cache state is not an implementation of Hugging Face `past_key_values`. Native cached generation and a framework's generation API are separate contracts. Preference workflows that call generation need a compatible cache adapter first. With MTP enabled, inspect the returned logits length as well as loss alignment.

Extending the attention zoo

`nano/attention_zoo.py` registers mixers behind a shared construction and forward interface. A new mixer must preserve residual shape, support its declared cache behavior, and expose/reset all retained state. Sharing a signature does not mean sharing the same internal memory representation.

Begin with the simplest full-sequence implementation. Add causality checks. Then implement incremental evaluation and compare it against that reference. Finally test boundaries and low-precision paths. Optimize only after you have a trustworthy reference to compare with. A fused kernel is harder to debug when the intended mathematical operation is still unclear.

A controlled capstone

Choose one question: does per-channel decay improve recall over per-head decay on your workload? Compare KDA and Gated DeltaNet with the same surrounding hybrid, data split, tokenizer, context length, and training-token budget. Account for

any parameter difference. Tune learning rates within a declared budget instead of quietly favoring one model.

Before training, run the relevant mechanical checks and save both resolved configs. During training, log the main LM loss separately from every auxiliary objective. Afterward, compare held-out loss and an order- or retrieval-sensitive task. Measure prefill, decode, and peak memory separately on the same hardware. Use several seeds and report their individual results; a single tiny-corpus loss difference is not evidence of a general architectural advantage.

Deliverable. Write one page containing the hypothesis, two source revisions/configs, data split, resource budget, correctness checks, measured results, and remaining uncertainty. A negative result that identifies a tradeoff is useful engineering work. Do not fill the result table until the runs exist.

What remains open

The [roadmap](#) distinguishes mechanism checks from quality evidence. Important extensions include validated distributed training, real padding support across recurrent/compressed mixers, generation adapters, streaming data, and sparse kernels that avoid dense work. They belong in future implementation work and measured revisions of this book. Vision and multimodal editions will add new representations, objectives, and evaluation contracts rather than simply adding more model names.

Repository atlas and paper trail

This atlas records every current model file, shared module, and preset. The linked implementation is the source of truth for defaults. The paper trail preserves research entry points from the companion documentation; it is not a claim that every production recipe or numerical result has been reproduced.

Source map

- nano/accel.py
- nano/attention_zoo.py
- nano/components.py
- nano/config.py
- nano/data.py
- nano/hf.py
- nano/optim.py
- nano/models/deepseek_nano.py
- nano/models/gemma_nano.py
- nano/models/gpt_nano.py
- nano/models/looped_nano.py
- nano/models/mamba_nano.py
- nano/models/qwen_nano.py
- nano/models/qwen_next_nano.py
- examples/sft_trl.py

Preset field values

These snapshots are read directly from `configs/` at build time. The model-family mapping appears in the recipes chapter. `_comment` fields are omitted from the code listings; their cautions are discussed there.

deepseek_v41_ced.json

```
{
  "size": "nano",
  "seed": 42,
  "model": {
    "n_layers": 8,
    "hybrid_ratio": 0,
    "attn": "gqa",
    "kv_sources": [
      null,
      null,
      null,
      null,
      0,
      1,
      2,
      3
    ],
    "engram_dim": 16,
    "engram_layer": 1,
    "mtp_weight": 0.3
  },
  "train": {
    "num_epochs": 40,
    "batch_size": 8,
    "learning_rate": 0.0003
  }
}
```

deepseek_v4_flash.json

```
{
  "size": "nano",
  "seed": 42,
  "model": {
    "hash_layers": 1,
    "router_score": "sqrtsoftplus",
    "balance_speed": 0.001,
    "swiglu_limit": 10.0
  },
  "train": {
    "num_epochs": 40,
    "batch_size": 8,
    "learning_rate": 0.0003
  }
}
```

deepseek_v4_like.json

```
{
  "size": "nano",
  "seed": 42,
  "model": {
    "linear_attn": "deltanet",
    "hybrid_ratio": 3,
    "residual": "mhc",
    "mhc_streams": 4,
    "mtp_weight": 0.3
  },
  "train": {
    "num_epochs": 40
  }
}
```

gemma4_like.json

```
{
  "size": "nano",
  "seed": 42,
  "model": {
    "hybrid_ratio": 1,
    "kv_share": 1,
    "ple_dim": 16,
    "mtp_weight": 0.0
  },
  "train": {
    "num_epochs": 40
  }
}
```

kimi_like.json

```
{
  "size": "nano",
  "seed": 42,
  "model": {
    "linear_attn": "kda",
    "attn": "mla",
    "hybrid_ratio": 3,
    "short_conv": 4,
    "mtp_weight": 0.0
  },
  "train": {
    "num_epochs": 40,
    "batch_size": 8,
    "learning_rate": 0.0003
  }
}
```

latent_moe.json

```
{
  "size": "nano",
  "seed": 42,
  "model": {
    "moe_latent_dim": 16,
    "balance_speed": 0.01
  },
  "train": {
    "num_epochs": 30
  }
}
```

qwen_flash_next.json

```
{
  "size": "nano",
  "seed": 42,
  "model": {
    "linear_attn": "deltanet",
    "hybrid_ratio": 3,
    "attn": "qsa",
    "qsa_block": 4,
    "qsa_top_k": 8,
    "engram_dim": 16,
    "engram_layer": 1,
    "mtp_weight": 0.3,
    "residual": "gr",
    "gr_branches": 4
  },
  "train": {
    "num_epochs": 40,
    "batch_size": 8,
    "learning_rate": 0.0003
  }
}
```

wikitext.json

```
{
  "size": "nano",
  "seed": 42,
  "data": {
    "dataset": "Salesforce/wikitext",
    "dataset_config": "wikitext-2-raw-v1",
    "text_field": "text",
    "split": "train",
    "dataset_limit": 2000
  },
  "model": {
    "linear_attn": "kda",
    "hybrid_ratio": 3,
    "mtp_weight": 0.0
  },
  "train": {
    "num_epochs": 1
  }
}
```

Research reading order

Read an original mechanism before a later model recipe. The entries below are sorted by arXiv identifier, whose prefix indicates its identifier month; this is not a fully audited release chronology. Several ideas have earlier precursors. The companion timeline distinguishes origins and later adoption, but release dates should be checked against the primary announcement before citation.

- **Tied input/output embeddings.** arXiv:1608.05859.
- **Sparse MoE, top-k routing.** arXiv:1701.06538.
- **Multi-head attention, the transformer.** arXiv:1706.03762.
- **Depth recurrence (looped layers).** arXiv:1807.03819.
- **RMSNorm.** arXiv:1910.07467.
- **Gated FFNs: SwiGLU, GeGLU.** arXiv:2002.05202.
- **Sliding-window attention.** arXiv:2004.05150.
- **Query-key normalization.** arXiv:2010.04245.
- **Sparse MoE, top-k routing.** arXiv:2101.03961.
- **RoPE.** arXiv:2104.09864.
- **Sandwich normalization.** arXiv:2105.13290.
- **Expert-choice routing.** arXiv:2202.09368.
- **NoPE.** arXiv:2203.16634.
- **Parallel attention + FFN block.** arXiv:2204.02311.
- **QK-norm at 22B scale (later adoption).** arXiv:2302.05442.
- **Grouped-query attention.** arXiv:2305.13245.
- **NoPE.** arXiv:2305.19466.
- **Lightning attention (linear, fixed decay).** arXiv:2307.14995.
- **Attention sinks (the diagnosis).** arXiv:2309.17453.
- **Sliding-window attention.** arXiv:2310.06825.
- **Lightning attention (linear, fixed decay).** arXiv:2401.04658.
- **Shared expert.** arXiv:2401.06066.
- **(1 + w) RMSNorm.** arXiv:2403.08295.
- **Mixture-of-Depths.** arXiv:2404.02258.
- **Multi-token prediction.** arXiv:2404.19737.
- **Multi-head latent attention.** arXiv:2405.04434.
- **Cross-layer KV sharing.** arXiv:2405.12981.
- **Mamba-2 / state-space duality.** arXiv:2405.21060.
- **Logit softcapping; RMSNorm sandwich adoption.** arXiv:2408.00118.
- **Aux-loss-free load balancing.** arXiv:2408.15664.
- **Hyper-connections.** arXiv:2409.19606.
- **Differential attention.** arXiv:2410.05258.
- **p-RoPE (partial RoPE).** arXiv:2410.06205.
- **Gated DeltaNet.** arXiv:2412.06464.
- **Sigmoid routing, MTP and aux-free balancing at scale.** arXiv:2412.19437.

- **Lightning attention at 456B.** [arXiv:2501.08313](#).
- **Scalable softmax.** [arXiv:2501.19399](#).
- **Depth recurrence revived.** [arXiv:2502.05171](#).
- **Mixture of block attention.** [arXiv:2502.13189](#).
- **Muon optimizer.** [arXiv:2502.16982](#).
- **5:1 local:global, dual RoPE base, QK-norm over softcap.** [arXiv:2503.19786](#).
- **MuonClip in production (related adoption).** [arXiv:2507.20534](#).
- **Depth recurrence revived.** [arXiv:2510.25741](#).
- **Kimi Delta Attention, KDA + gated MLA.** [arXiv:2510.26692](#).
- **Short convolutions as a mixer.** [arXiv:2511.23404](#).
- **DeepSeek sparse attention, lightning indexer.** [arXiv:2512.02556](#).
- **Mamba-2 + attention + MoE hybrid.** [arXiv:2512.20856](#).
- **Manifold-constrained hyper-connections.** [arXiv:2512.24880](#).
- **Engram conditional memory.** [arXiv:2601.07372](#).
- **Mamba-3: trapezoidal step, complex state.** [arXiv:2603.15569](#).
- **LatentMoE.** [arXiv:2604.12374](#).
- **Short conv vs. short window, the open argument.** [arXiv:2606.03825](#).
- **IndexShare: one indexer per layer group.** [arXiv:2606.13392](#).
- **Lightning attention + MLA hybrid.** [arXiv:2606.15079](#).
- **Compressed sparse / heavily compressed attention; mHC shipped.** [arXiv:2606.19348](#).
- **K reused as V, p-RoPE and PLE shipped, KV sharing, last layer always global.** [arXiv:2607.02770](#).
- **3:1 GDN + Qwen Sparse Attention (4-token micro-block indexer, top-512 blocks), 51B n-gram embedding at layer 2, 512-expert MoE top-10, Gated Residual (4 branches, no mixing matrix).** [arXiv:2608.30320](#).

Edition provenance and credits

Author: Ayoub Kirouane. [Portfolio](#). The architecture chapters are edited adaptations of the repository guide, expanded with engineering exercises and implementation boundaries. No original benchmark results were generated for this edition.

The original latte-themed visual guides and component diagrams use a custom palette and the rendering/checking tools from [OrRon/EpicInfographics](#), MIT license. The pinned commit and local adaptation are recorded in the book tools directory. Figures show mechanisms or stated arithmetic, not observed speedups.

This edition covers language models. Later editions will add vision and multimodal material when their implementations, source readings, and exercises exist.

Corrections should identify the affected edition and companion-code revision.

Public releases should preserve older PDFs rather than replacing them silently.